



Deep learning for solving dynamic economic models.

Lilia Maliar^a, Serguei Maliar^{b,*}, Pablo Winant^c

^a The Graduate Center, City University of New York, CEPR, and Hoover Institution, Stanford University

^b Santa Clara University

^c ESCP Business School and CREST/Ecole Polytechnique



ARTICLE INFO

Article history:

Received 11 March 2020

Revised 16 July 2021

Accepted 19 July 2021

Available online 21 July 2021

JEL classification:

C61

C63

C65

C68

C88

E32

E37

Keywords:

Artificial intelligence

Machine learning

Deep learning

Neural network

Stochastic gradient

Dynamic models

Model reduction

Dynamic programming

Bellman equation

Euler equation

Value function

ABSTRACT

We introduce a unified deep learning method that solves dynamic economic models by casting them into nonlinear regression equations. We derive such equations for three fundamental objects of economic dynamics – lifetime reward functions, Bellman equations and Euler equations. We estimate the decision functions on simulated data using a stochastic gradient descent method. We introduce an all-in-one integration operator that facilitates approximation of high-dimensional integrals. We use neural networks to perform model reduction and to handle multicollinearity. Our deep learning method is tractable in large-scale problems, e.g., Krusell and Smith (1998). We provide a TensorFlow code that accommodates a variety of applications.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

Artificial intelligence (AI) has remarkable applications, such as recognition of images and speech, facilitation of computer vision, operation of self-driving cars; see Goodfellow et al. (2016) for a review. At the same time, there are many interesting problems that computational economists cannot solve yet, including high-dimensional heterogeneous-agent models, large-scale central banking models, life-cycle models, and expensive nonlinear estimation procedures, among others. We show

* Corresponding author.

E-mail address: maliars@stanford.edu (S. Maliar).

that it is possible to solve many challenging economic models by using the same AI technology, software and hardware that led to groundbreaking applications in data science. We specifically introduce an econometric-style deep learning (DL) method that solves dynamic economic models by reformulating them as nonlinear regression equations. Our four novel results are stated below:

First, we offer a *unified approach* which allows us to cast three fundamental objects of economic dynamics – *lifetime reward functions*, *Bellman equations* and *Euler equations* – into objective functions for Monte Carlo simulation. Such objective functions are given by a weighted sum of all of the model's equations, so we iterate on the entire model and solve for all decision functions at once. To optimize the constructed objective functions, we use deep learning regression techniques from the fields of econometrics and data science. Once the regression coefficients are constructed, we infer the value and decision functions of the underlying dynamic economic models.

Second, we show how to adapt a *stochastic gradient descent method* to training of the three constructed objective functions. In each iteration, we use just one or a few (batch) grid points, which are randomly drawn from the state space, instead of a fixed grid with a large number of grid points used by conventional projection and value iterative methods. In small problems, we draw grid points from an exogenous solution domain, but in large problems, we produce grid points by stochastic simulation which allows us to focus on the ergodic set in which the solution "lives", avoiding the cost of computing solutions in those areas that are never visited in equilibrium. Thus, our DL framework aims not only on convergence of decision and value functions along iterations but also on convergence of simulated series to the ergodic set.

Third, we introduce the *all-in-one (AiO) expectation operator* for efficient approximation of integrals in Monte Carlo simulation. The objective functions, which we derive from economic models, have two types of expectation operators. One is with respect to next-period shocks (which appears naturally in stochastic models), and the other is with respect to the current state variables (which we created ourselves by drawing grid points randomly from the state space). Approximating these two nested expectation operators is costly, especially, in large-scale applications. The AiO method merges the two expectation operators into one, reducing the cost dramatically. It possesses a remarkable distributive property: a single composite Monte Carlo draw is used both for integration with respect to future shocks and for approximation of decision functions.

The way we construct the AiO operator differs for the three objective functions considered. For the lifetime reward method, we draw randomly the initial condition, in addition to future shocks. For the Euler-equation method, we use two independent random draws (or two independent batches) for evaluating two terms of the squared residual – this method eliminates the correlation between the two terms and helps us pull the expectation operator out of the square. Finally, for the Bellman-equation method, we introduce a value-iterative scheme that combines a minimization of residuals in the Bellman equation with a maximization of the right side of the Bellman equation into a single weighted-sum objective function. We use the Fischer-Burmeister function for a smooth approximation of Kuhn-Tucker conditions.

Our last important contribution is to implement the DL solution framework using the Google TensorFlow data platform – the same software that lead to ground-breaking applications in data science. Our implementation is versatile and portable to a variety of economic models and applications.³

The solution framework we introduce is not tied to neural networks but can be used with any approximating family (e.g., polynomials, splines, radial basis functions). However, neural networks possess several features that make them an excellent match for high-dimensional applications; namely, they are linearly scalable, robust to ill-conditioning, capable of model reduction and well suited for approximating highly nonlinear environments including kinks, discontinuities, discrete choices, switching.

We first illustrate our DL solution framework by using a simple one-agent consumption savings problem with a borrowing constraint. We implement three versions of the deep learning method based on *lifetime reward*, *Bellman equation* and *Euler equation* – they produce very similar solutions. Approximation errors do not exceed a fraction of a percentage point – an impressive accuracy level for a model with a kink in decision rules! Moreover, the computational expense increases practically linearly with the dimensionality of the state space – another outstanding feature of our DL method based on stochastic gradient and the AiO integration operator.

We then solve [Krusell and Smith \(1998\)](#) model with heterogeneous agents. Our solution procedure is conceptually straightforward – we simulate a panel of heterogeneous agents, and we feed a distribution of labor productivity and wealth into the constructed objective functions for training. But there are two challenges: First, the decision function of each agent depends on the state variables of all agents, which makes the problem high dimensional. Second, the agent's state variables appear twice in the decision function (as agent's own state variables and as a part of the distribution), which leads to perfect collinearity. Fortunately, a neural network can deal with these challenges: First, it performs model reduction by extracting and condensing information from high-dimensional distributions into a smaller set of features of the hidden layers. Second, it learns to ignore the presence of redundant collinear variables. We again implement deep learning methods based on lifetime reward, Bellman equation and Euler equation and show that they produce very similar solutions. Our solution method is tractable in models with at least 1000 agents (2,001 state variables) on a serial desktop computer!

³ Jupyter notebooks illustrating the method are available from open-source QuantEcon.org site <https://notes.quantecon.org/submission/5ddb3c926bad3800109084bf>.

We next propose a cheaper deep learning method that replaces the actual state space composed of distributions with a reduced state space composed of some aggregate statistics such the moments of wealth distribution studied in [Krusell and Smith \(1998\)](#). Implementing such a method requires no modifications: as before, we simulate a panel of heterogeneous agents, but we now feed in moments instead of distributions. In contrast, the method proposed by [Krusell and Smith \(1998\)](#) is far more complicated: they alternate between constructing individual and aggregate decision rules. Also, they rely on a regression of current moments on past moments, which is unnecessary in our case. Having relatively few moments, like 10 or 20, implies lower computational expense and allows us to increase the number of agents (at least) to 10,000 agents without a visible accuracy loss, so this cheaper method is a useful alternative to our baseline method.

We finally compare the solutions constructed with actual state space to those produced with a reduced state space. We find that the solution constructed by using the first moment of wealth distribution as in [Krusell and Smith \(1998\)](#) is somewhat shifted up relatively to our baseline solution. We tried to add second and third moments, but it did not help remove the shift. We then constructed a solution with the actual state space but using only 4 neurons which is parallel to 4 state variables in [Krusell and Smith \(1998\)](#) method with one moment. We find that the 4-neuron solution is also shifted up near the kink area but the shift reduces for larger wealth levels. Furthermore, we find that having more neurons helps one get closer to the reference solution, unlike having more moments. These findings suggest that moments are not the best reduced representation of the actual state space which is not surprising given that the moments are selected by a guess, while neural networks are designed to search for the best possible reduced representation.

Our solution method is related to supervised-learning (because we fit the decision and value functions to the data which are artificial in our analysis), to unsupervised learning (because the decision and value functions are not explicitly labeled) and reinforcement learning (because we attempt to achieve the convergence of simulated series to the ergodic set, in addition to convergence of the neural network coefficients). We are not the only paper that uses machine learning tools for analyzing dynamic economic models. There are numerous methods that solve dynamic economic models on their ergodic sets approximated via stochastic simulation, such as the indirect inference procedure of [Smith \(1987\)](#) for maximizing the lifetime reward, a parameterized expectation algorithm (PEA) by [Den Haan and Marcet \(1990\)](#) for minimizing the Euler equation residuals and a value iterative method of [Maliar and Maliar \(2005\)](#) for minimizing the Bellman equation residuals. There are also methods that use unsupervised learning in order to aim to refine simulated points and determine the irregularly-shaped ergodic sets. In particular, [Judd et al. \(2011\)](#) uses clustering of simulated points, [Maliar and Maliar \(2015\)](#) combine simulated points in epsilon-distinguishable sets, [Renner and Scheidegger \(2018\)](#) and [Scheidegger and Bilonis \(2019\)](#) use Gaussian process machine learning to identify feasible sets. In turn, [Jirniy and Lepetyuk \(2011\)](#) show an early remarkable application of reinforcement learning for solving [Krusell and Smith \(1998\)](#) model.

Furthermore, machine learning methods for model reduction and dealing with ill-conditioning are analyzed in [Judd et al. \(2011\)](#) including a principle component regression, a truncated SVD method, Tykhonov regularization and regularized least absolute deviation methods. The other methods that use model reduction for solving heterogeneous-agent models are [Ahn et al. \(2018\)](#); [Reiter \(2010\)](#); [Winberry \(2018\)](#) and [Bayer and Luetticke \(2020\)](#).

Finally, early applications of neural networks date back to [Duffy and McNelis \(2001\)](#) and more recent applications include [Duarte \(2018\)](#); [Fernández-Villaverde et al. \(2019\)](#); [Lepetyuk et al. \(2020\)](#); [Villa and Valaitis \(2019\)](#). These papers use neural networks for interpolation instead of polynomial functions. To the best of our knowledge, we are the first to cast an entire economic model into the state-of-the-art DL framework and to construct a solution on simulated points by using stochastic gradient descent method. There is also a paper by [Azinovic et al. \(2020\)](#) that uses a related Euler-equation method to solve a large-scale OLG problem. Like us, that paper uses deep neural network and random grid points but focuses only on the method that minimizes the Euler equation residuals while we offer a unified approach that applies also to the lifetime reward and Bellman operator. Another difference is that [Azinovic et al. \(2020\)](#) assume a finite number of shocks in which case integration is exact, while we show how to integrate stochastic processes with continuous transition density by using the AiO operator – a key contribution of our analysis. Finally, the techniques we developed in the present paper are used in [Maliar and Maliar \(2020\)](#) for constructing a classification deep learning method for modeling non-convex labor choices, and in [Gorodnichenko et al. \(2020\)](#) for solving a version of heterogeneous-agent new Keynesian model with uncertainty shocks.

The rest of the paper is organized as follows: [Section 2](#) shows how to cast three main objects of economic dynamics (lifetime reward, Bellman equation and Euler equations) into expectation functions. [Section 3](#) presents a deep learning solution method and provides a quick overview of its key ingredients (multilayer neural networks, stochastic gradient training method, etc.). [Sections 4](#) and [5](#) analyze the one-agent consumption-saving model and [Krusell and Smith \(1998\)](#) heterogeneous-agent model, respectively. Finally, [Section 7](#) concludes.

2. Casting dynamic economic models into DL expectation functions

Deep learning platforms such as TensorFlow or PyTorch provide efficient ways of numerically approximating expectation functions with large numbers of parameters. In this section, we show how to cast dynamic economic models into the form of expectation functions that can be suitable for deep learning platforms. Specifically, we show how to reformulate as expectation functions three key objects of economic dynamics: lifetime reward, Euler equation and Bellman equation.

2.1. A class of dynamic economic models

We consider a class of dynamic Markov economic models with time-invariant decision functions – the main framework in modern economic dynamics. An agent (consumer, firm, government, central bank, etc.) solves a canonical intertemporal optimization problem.⁴

Definition 2.1 (Optimization problem). *An exogenous state $m_{t+1} \in \mathbb{R}^m$ follows a Markov process driven by an i.i.d. innovation process $\epsilon_t \in \mathbb{R}^m$ with a transition function M ,*

$$m_{t+1} = M(m_t, \epsilon_t). \quad (1)$$

An endogenous state s_{t+1} is driven by the exogenous state m_t and controlled by a choice $x_t \in \mathbb{R}^{n_x}$ according to a transition function S ,

$$s_{t+1} = S(m_t, s_t, x_t, m_{t+1}). \quad (2)$$

The choice x_t satisfies the constraint in the form

$$x_t \in X(m_t, s_t). \quad (3)$$

The state (m_t, s_t) and choice x_t determine the period reward $r(m_t, s_t, x_t)$. The agent maximizes discounted lifetime reward

$$\max_{\{x_t, s_{t+1}\}_{t=0}^{\infty}} E_0 \left[\sum_{t=0}^{\infty} \beta^t r(m_t, s_t, x_t) \right], \quad (4)$$

where $\beta \in [0, 1)$ is the discount factor and $E_0[\cdot]$ is an expectation function across future shocks $(\epsilon_1, \epsilon_2, \dots)$ conditional on the initial state (m_0, s_0) .

Without loss of generality, we assume that the constrained sets are re-mapped into a set of real numbers, so that the transition and reward functions are defined for any succession of choices $x_t \in \mathbb{R}^{n_x}$. We focus on recursive Markov time-invariant solutions.

Definition 2.2 (Decision rules). *i) An optimal decision rule is a function $\varphi : \mathbb{R}^m \times \mathbb{R}^{n_s} \rightarrow \mathbb{R}^{n_x}$ such that $x_t = \varphi(m_t, s_t) \in X(m_t, s_t)$ for all t and the sequence $\{x_t, s_{t+1}\}_{t=0}^{\infty}$ maximizes the lifetime reward (4) for any initial condition (m_0, s_0) ii) A parametric decision rule is a member of a family of functions $\varphi(\cdot; \theta)$ parameterized by a real vector $\theta \in \Theta$ such that for each θ , we have $\varphi : \mathbb{R}^m \times \mathbb{R}^{n_s} \rightarrow \mathbb{R}^{n_x}$ and $x_t = \varphi(m_t, s_t) \in X(m_t, s_t)$ for all t .*

Our goal is to find a vector of parameters $\theta \in \Theta$ under which the parametric decision rule $\varphi(\cdot; \theta)$ provides an accurate approximation of the optimal decision rule φ on a relevant domain. We do not assume a smoothness of the approximation function $\varphi(\cdot; \theta)$ nor its linearity with respect to coefficients θ and state (m_t, s_t) . But we do require the problem to be time consistent, so that its solving amounts to finding time-invariant decision rules.

2.2. Objective 1: Lifetime-reward maximization

We first introduce a method that maximizes the lifetime reward (4) directly.

Definition 2.3 (Value function). *For a given distribution of shocks $(\epsilon_1, \dots, \epsilon_T)$, value function $V(m_0, s_0)$ is a maximum expected lifetime reward (4) that is attainable from a given initial condition (m_0, s_0) :*

$$V(m_0, s_0) \equiv \max_{\{x_t, s_{t+1}\}_{t=0}^{\infty}} E_{(\epsilon_1, \dots, \epsilon_T)} \left[\sum_{t=0}^{\infty} \beta^t r(m_t, s_t, x_t) \right], \quad (5)$$

where transitions are determined by equations (1), (2) and (3).

For numerical approximation of V , we replace the infinite-horizon problem with a finite-horizon problem by truncating it at some finite $T < \infty$. We then simulate time series solution forward under a fixed decision rule $\varphi(\cdot; \theta)$ and evaluate the lifetime reward:

$$V^T(m_0, s_0; \theta) \equiv E_{(\epsilon_1, \dots, \epsilon_T)} \left[\sum_{t=0}^T \beta^t r(m_t, s_t, \varphi(m_t, s_t; \theta)) \right]. \quad (6)$$

Our first method constructs approximation $\varphi(\cdot; \theta)$ to the optimal decision rule by searching for a vector of coefficients θ that maximizes the lifetime reward (6).

⁴ A general model formulation in this paper matches standard API used by modeling software Dolo available at <https://github.com/econforge/dolo>. This makes it easily feasible to compare various deep-learning approaches described here with more traditional iterative methods already implemented in Dolo. We leave it for further work.

A potential shortcoming of the objective function (6) depends on a specific initial condition (m_0, s_0) . If we always start simulation from the same initial condition, we get an accurate approximation in a neighborhood of this specific initial condition but not for the states further away from this initial condition. Although the simulated series $\{(m_t, s_t)\}_{t=0}^T$ may pass many values, the contribution of future utility levels to the lifetime reward decreases with time due to discounting, so the initial condition still dominates accuracy. A possible way to achieve high accuracy on a larger domain would be to construct a solution on a grid of initial conditions $\{(m_0, s_0)\}$. However, here, we propose an alternative approach which is more suitable for Monte Carlo simulation implemented by deep learning tools, namely, we reformulate (6) as an expectation function. Instead of a fixed grid, we assume that initial condition (m_0, s_0) is drawn randomly from the domain on which we want the solution to be accurate, which yields the following objective function:

$$\Xi(\theta) \equiv E_{(m_0, s_0)} \left\{ E_{(\epsilon_1, \dots, \epsilon_T)} \left[\sum_{t=0}^T \beta^t r(m_t, s_t, \varphi(m_t, s_t; \theta)) \right] \right\}. \quad (7)$$

By solving $\max_{\theta \in \Theta} \Xi(\theta)$, we construct a decision rule $\varphi(\cdot; \theta)$ that maximizes the lifetime reward for a given distribution of initial conditions.

A new feature of the objective function $\Xi(\theta)$ is that it has two types of randomness: one is a random sequence of future shocks $(\epsilon_1, \dots, \epsilon_T)$, which appears because the model is stochastic, and the other is a random state (m_0, s_0) , which we created ourselves because we converted the initial condition into a random variable. Approximating two nested expectation operators, one after the other, is costly, especially in high dimensional applications. That is, if we make n draws for evaluating expectation with respect to (m_0, s_0) and if we make n' draws for evaluating expectation with respect to $(\epsilon_1, \dots, \epsilon_T)$, in total, we must evaluate $n \times n'$ draws.

To reduce the cost of nested integration, we introduce *all-in-one* (AiO) expectation operator that combines the two expectation operators into one.

Definition 2.4 (All-in-one expectation operator for lifetime reward). Fix time horizon $T > 0$, parametrize a decision rule $\varphi(\cdot; \theta)$ and define the distribution of the random variable $\omega \equiv (m_0, s_0, \epsilon_1, \dots, \epsilon_T)$. For given θ , lifetime reward (4) associated with the rule $\varphi(\cdot; \theta)$ is given by

$$\Xi(\theta) = E_{\omega}[\xi(\omega; \theta)] \equiv E_{(m_0, s_0, \epsilon_1, \dots, \epsilon_T)} \left[\sum_{t=0}^T \beta^t r(m_t, s_t, \varphi(m_t, s_t; \theta)) \right], \quad (8)$$

where transitions are determined by equations (1), (2) and (3), and ξ is an integrand.

The AiO operator can significantly reduce the cost of evaluation expectations. Instead of making $n \times n'$ draws for the two random vectors (m_0, s_0) and $(\epsilon_1, \dots, \epsilon_T)$, we make just n draws for a composite random variable $(m_0, s_0, \epsilon_1, \dots, \epsilon_T)$. Constructing the AiO operator is easy for the lifetime reward maximization studied in this section but it will be more challenging for the Euler and Bellman methods studied in next sections.

2.3. Objective 2: Euler-residual minimization

We next introduce a DL method that constructs a solution to the Euler equations. We consider a class of economic models in which the objective functions are differentiable, so that the solution is characterized by a set of first-order conditions (Euler equations). Such equations may follow from an optimal control problem of type (4) or from an equilibrium problem and may include first-order conditions, equilibrium conditions, transition equations, constraints, market clearing conditions, etc.

Definition 2.5 (Euler equations). Euler equations are a set of equations written in the form:

$$E_{\epsilon} [f_j(m, s, x, m', s', x')] = 0, \quad j = 1, \dots, J, \quad (9)$$

where the agent's choice satisfies constraints (1), (2) and (3) expressed in a recursive form $m' = M(m, \epsilon)$, $s' = S(m, s, x, m')$ and $x \in X(m, s)$, respectively; $f_j : \mathbb{R}^{n_m} \times \mathbb{R}^{n_s} \times \mathbb{R}^{n_x} \times \mathbb{R}^{n_m} \times \mathbb{R}^{n_s} \times \mathbb{R}^{n_x} \rightarrow \mathbb{R}$ and $E_{\epsilon}[\cdot]$ is an expectation operator with respect to the next-period shock ϵ .

Equations (9) are again defined just for a given state (m, s) . The typical approach in computational economics is to solve the Euler equation on a fixed grid that covers a relevant area of the state space. Like with lifetime reward, we do not follow this approach but assume that states (m, s) are drawn randomly from a given distribution. The corresponding objective function is defined as an expected squared sum of residuals in the Euler equations for a given distribution of states.

Definition 2.6 (Euler-residual minimization). Select a decision rule $\varphi(\cdot; \theta)$, and define a distribution of random variable (m, s) . For given θ , the expected squared residuals in the Euler equations (9) associated with the rule $\varphi(\cdot; \theta)$ are given by

$$\Xi(\theta) = E_{(m, s)} \left\{ \sum_{j=1}^J v_j (E_{\epsilon} [f_j(m, s, \varphi(m, s; \theta), m', s', \varphi(m', s'; \theta))])^2 \right\}. \quad (10)$$

where $(v_1, \dots, v_J)$ is a vector of weights on J optimality conditions.

Our goal is to construct a decision rule $\varphi(\cdot; \theta)$ that solves $\max_{\theta \in \Theta} \Xi(\theta)$.

Again, the objective function (10) has also two expectation operators, one with respect to the shocks $E_\epsilon[\cdot]$ and the other is with respect to the state $E_{(m,s)}[\cdot]$. In the case of lifetime reward maximization, combining the two expectation operators was easy because the expectation operators enter the objective linearly, so the AiO operator just merges them together, i.e., $E_{(m_0, s_0)}[E_{(\epsilon_1, \dots, \epsilon_T)} r(\cdot)] = E_{(m_0, s_0, \epsilon_1, \dots, \epsilon_T)}[r(\cdot)]$. However, in the Euler equations, $E_\epsilon[\cdot]$ is squared, so the two expectations cannot be naturally merged since $E_{(m,s)}(E_\epsilon[f_j(m, s, \epsilon)])^2 \neq E_{ms} E_\epsilon[f_j(m, s, \epsilon)^2]$.

An important contribution of the present paper is to offer a technique that allows us to combine the expectation functions $E_{m,s}[\cdot]$ and $E_\epsilon[\cdot]$ in the AiO expectation operator in the presence of squares. The technique is very simple but effective, namely, instead of using the same random draw ϵ for both terms in the square, we use two independent random draws or two independent batches ϵ_1 and ϵ_2 which yields

$$E_{\epsilon_1}[f(\epsilon_1)]E_{\epsilon_2}[f(\epsilon_2)] = E_{(\epsilon_1, \epsilon_2)}[f(\epsilon_1)f(\epsilon_2)]. \quad (11)$$

With this approach, we are able to write the Euler-residual function (10) as an expectation function $E_{ms}E_{\epsilon_1\epsilon_2}[\cdot]$ of a single random vector.

Definition 2.7 (Euler-residual minimization with all-in-one expectation operator). Parametrize a decision rule $\varphi(\cdot; \theta)$, and define a distribution of random variable $\omega \equiv (m, s, \epsilon_1, \epsilon_2)$. For a given θ , the squared residuals in the Euler equations (9) associated with the rule $\varphi(\cdot; \theta)$ are given by

$$\Xi(\theta) = E_\omega[\xi(\omega; \theta)] \equiv E_{(m,s,\epsilon_1,\epsilon_2)} \left\{ \sum_{j=1}^J v_j \left[f_j(m, s, x, m', s', x') \Big|_{\epsilon=\epsilon_1} \right] \left[f_j(m, s, x, m', s', x') \Big|_{\epsilon=\epsilon_2} \right] \right\}, \quad (12)$$

where $(v_1, \dots, v_J)$ is a vector of weights on J optimality conditions.

The method based on the AiO expectation operator is our main method. However, we also develop and test various hybrid methods that construct two expectations separately. For example, we use a Monte Carlo method for constructing $E_{(m,s)}[\cdot]$, and we use some other methods for constructing $E_\epsilon[\cdot]$ such as quadrature and monomial rules, sparse grids, low-discrepancy sequences – we show these sensitivity results in Section 5. Overall, we find that such hybrid methods are useful for small problems in which the construction of $E_\epsilon[\cdot]$ is inexpensive but the AiO expectation operator is critical for large problems, such as Krusell and Smith (1998) model studied in Section 6.

Objective 3: Bellman-residual minimization

Our last DL method constructs the decision rule to satisfy the Bellman equation.

Definition 2.8 (Bellman equation). Value function $V : \mathbb{R}^{nm} \times \mathbb{R}^{ns} \rightarrow \mathbb{R}$ associated with the problem (4) satisfies:

$$V(m, s) = \max_{x, s'} \{ r(m, s, x) + \beta E_\epsilon[V(m', s')] \}, \quad (13)$$

subject to constraints (1), (2) and (3) expressed in a recursive form $m' = M(m, \epsilon)$, $s' = S(m, s, x, m')$ and $x \in X(m, s)$.

Under the standard assumptions about r , M , S and X , the solution to (13) exists and is unique. Similar to the Euler-equation method, we can find an approximate decision rule $\varphi(\cdot; \theta)$ by minimizing the squared residuals in the Bellman equation (13) on a conventional fixed grid but in the spirit of deep learning, we will make (m, s) a random variable which is drawn from a given distribution.

In the case of Bellman operator, we face an additional element – a nontrivial “max” operator appears inside the squared residuals:

$$\Xi(\theta) = E_{(m,s)} \left[V(m, s) - \max_{x, s'} \{ r(m, s, x) + \beta E_\epsilon[V(m', s')] \} \right]^2.$$

There are three approaches in the literature for constructing a solution to the maximum operator, namely, the FOCs, the envelope condition and a direct search of maximum:

$$\begin{aligned} \text{FOC:} & \quad r_x(m, s, x) + \beta \{ E_\epsilon[V_{s'}(m', s')] \} \frac{\partial s'}{\partial x} = 0, \\ \text{Envelope condition:} & \quad r_s(m, s, x) = V_s(m, s), \\ \text{Direct optimization:} & \quad \max_{x, s'} \{ r(m, s, x) + \beta E_\epsilon[V(m', s')] \}. \end{aligned}$$

FOCs and direct optimization are used in conventional value function iteration, e.g., Aruoba et al. (2006); Rust et al. (1996); Santos et al. (1999); Stachurski (2009) while the envelope condition method is introduced in Maliar and Maliar (2013) and

developed in [Arellano et al. \(2016\)](#).⁵ Provided that any of these three conditions is enforced, we can eliminate the maximum operator from the Bellman equation. Consequently, we can formulate an objective function that solves for both value function and decision rule by combining minimization of the residuals in the Bellman equation with maximization of the right side of the Bellman equation. We focus on the FOC but the other two conditions can be treated in a similar way.⁶

Definition 2.9 (Bellman-residual minimization). *Parametrize a value function $V(\cdot; \theta_1)$ and decision rule $x = \varphi(\cdot; \theta_2)$ and define a distribution of the random variable $\omega \equiv (m, s)$. For given $\theta \equiv (\theta_1, \theta_2)$, the squared residuals in the Bellman equations (13) associated with $V(\cdot; \theta_1)$ and $\varphi(\cdot; \theta_2)$ are given by*

$$\Xi(\theta) \equiv E_{(m,s)} \left\{ \left[V(m, s; \theta_1) - r(m, s, x) - \beta E_\epsilon [V(m', s'; \theta_1)] \right]^2 + v E_{(m,s)} \left\{ r_x(m, s, x) + \beta \left\{ E_\epsilon [V_{s'}(m', s'; \theta_1)] \right\} \frac{\partial s'}{\partial x} \right\} \right\}, \quad (14)$$

where $v > 0$ is a vector of exogenous relative weights of equations in the two objectives.

Similar to the previous methods, the objective function for the Bellman equation has two expectation operators under the square. One expectation is taken with respect to the shocks $E_\epsilon[\cdot]$ and the other is with respect to the state $E_{(m,s)}[\cdot]$. Fortunately, we again can use the method of uncorrelated shocks (11) for constructing the AiO expectation operator.

Definition 2.10 (Bellman-residual minimization with all-in-one expectation operator). *Select value function $V(\cdot; \theta_1)$ and decision rule $x = \varphi(\cdot; \theta_2)$ and define the distribution of the random variable $\omega \equiv (m, s)$. For given $\theta \equiv (\theta_1, \theta_2)$, the squared residuals in the Bellman equations (13) associated with $V(\cdot; \theta_1)$ and $\varphi(\cdot; \theta_2)$ are given by*

$$\begin{aligned} \Xi(\theta) = E_\omega [\xi(\omega; \theta)] &\equiv E_{(m,s,\epsilon_1,\epsilon_2)} \left\{ \left[V(m, s; \theta_1) - r(m, s, x) - \beta V(m', s'; \theta_1) \right]_{\epsilon=\epsilon_1} \right. \\ &\quad \times \left[V(m, s; \theta_1) - r(m, s, x) - \beta V(m', s'; \theta_1) \right]_{\epsilon=\epsilon_2} \\ &\quad \left. + v \left[r_x(m, s, x) + \beta V_{s'}(m', s'; \theta_1) \right]_{\epsilon=\epsilon_1} \frac{\partial s'}{\partial x} \right] \left[r_x(m, s, x) + \beta V_{s'}(m', s'; \theta_1) \right]_{\epsilon=\epsilon_2} \frac{\partial s'}{\partial x} \right\}, \end{aligned} \quad (15)$$

where $v > 0$ is a vector of exogenous relative weights of equations in the two objectives.

Like for the Euler equation, our main Bellman-equation method is the one based on AiO expectation operator but we could also construct hybrid methods based on the objective (14) that would combine Monte Carlo integration for the state space with deterministic integration across future shocks.

3. Deep learning solution method

In each of the considered cases (lifetime reward, Euler and Bellman equations), we represent an economic model as a problem of minimizing an objective function $\Xi(\theta)$ with respect to a vector of parameters θ :

$$\min_{\theta \in \Theta} \Xi(\theta) = \min_{\theta \in \Theta} E_\omega [\xi(\omega; \theta)], \quad (16)$$

where $\omega \equiv (m, s, \epsilon)$ includes exogenous state variables m , endogenous state variables s and future shocks ϵ . By construction, $\Xi(\theta)$ contains all model's equations (Euler and Bellman equations, constraints, market clearing conditions, transition equations, multipliers, prices), so by minimizing a single objective function, we solve the entire model.

In computational economics, a common approach to solving dynamic economic models is to use a fixed grid of points in the state space (m, s) and to approximate expectation functions over future shock ϵ with quadrature nodes, see, e.g., a projection method of [Judd \(1992\)](#). A distinctive feature of our analysis is that we interpret (16) not as a computational problem but as an estimation / regression model studied in the fields of econometrics and machine learning. In particular, we treat ω as a vector of random variables, and we make no distinction between its components (m, s, ϵ) . We simulate the model to produce a set of random draws $\{\omega_i\}_{i=1}^n$, and we replace the expected risk $\Xi(\theta)$ with empirical risk $\Xi^n(\theta)$ – the sample average of ξ across n random draws – to obtain the following nonlinear regression model:

$$\min_{\theta \in \Theta} \Xi^n(\theta) = \min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n \xi(\omega_i; \theta). \quad (17)$$

We construct a solution θ by training the machine to minimize the empirical risk $\Xi^n(\theta)$ on the simulated data. Note that our data $\{\omega_i\}_{i=1}^n$ are being constantly re-sampled during the training process, unlike the data in a typical regression model that are assumed to be fixed. As a result, a successful training means two types of convergence: first, the parameter vector

⁵ There is also a method of reformulating state space in terms of the future endogenous state variables by [Carroll \(2006\)](#), which is known as endogenous-grid method. It is straightforward to generalize the proposed techniques to include this method as well.

⁶ In the earlier version of the paper, we study a version of the method that relies on direct search of a maximum.

θ converges to a value that minimizes an objective function $\Xi^n(\theta)$ on simulated data $\{\omega_i\}_{i=1}^n$; and second, the simulated data themselves, generated via the decision rule $\varphi(\cdot; \theta)$, converge to the ergodic set. While our optimization problem is not equivalent to the typical data science application, we can still solve it by using the same combination of techniques that led to ground breaking applications in data science. Those techniques include deep neural networks, Monte Carlo simulation and stochastic optimization and they are discussed below in the context of a numerical method for solving dynamic economic models. In Supplement D, we discuss how our solution method is related to supervised, unsupervised and reinforcement learning literature.

Neural network: effective approximation on unstructured data. We approximate the decision rules and value function with neural networks instead of conventional polynomial functions. Neural networks possess several properties that make them preferable to polynomial functions in high dimensional applications with unstructured data; namely, they are: (i) linearly scalable, i.e., the number of parameters grows linearly with dimensionality; (ii) robust to multicollinearity and can automatically perform model reduction; and (iii) well suited for fitting highly nonlinear environments including kinks, discontinuities, discrete choices, and switching.

A neural network is a collection of connected nodes – artificial neurons. Each neuron receives a signal (input) from other neurons, processes it and transmits the processed signal to some other neurons connected to it. In Fig. 1, we show an example of neural network with three layers – an input layer, hidden layer and output layer.

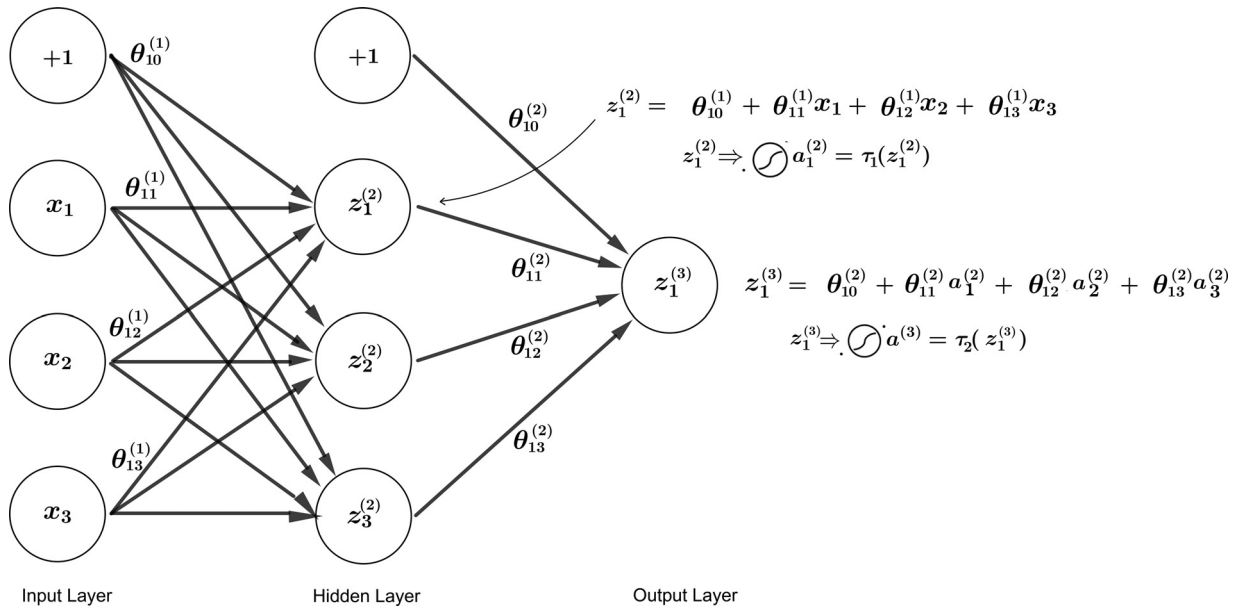


Fig. 1. A neural network with one hidden layer.

The input layer consists of a constant term $+1$ and input features (x_1, x_2, x_3) which correspond to state variables of an economic model. In that layer, we construct linear polynomial functions $z_1^{(2)}, z_2^{(2)}$ and $z_3^{(2)}$ on given inputs, for example, $z_1^{(2)} = \theta_{10}^{(1)} + \theta_{11}^{(1)}x_1 + \theta_{12}^{(1)}x_2 + \theta_{13}^{(1)}x_3$, where the coefficient on a constant term $\theta_{10}^{(1)}$ is called a bias, and the coefficients on features $\theta_{11}^{(1)}, \theta_{12}^{(1)}$ and $\theta_{13}^{(1)}$ are called weights. We next pass $z_1^{(2)}, z_2^{(2)}$ and $z_3^{(2)}$ to the hidden layer, where we apply to them a transformation τ_1 , such as a sigmoid (logistic) activation function $\tau_1(x) = \frac{1}{1+e^{-x}}$.⁷ Finally, in the output layer, we combine the activated signals $a_1^{(2)} = \tau_1(z_1^{(2)})$, $a_2^{(2)} = \tau_1(z_2^{(2)})$ and $a_3^{(2)} = \tau_1(z_3^{(2)})$ with a constant term $+1$ into a new polynomial function $z_1^{(3)} = \theta_{10}^{(2)} + \theta_{11}^{(2)}a_1^{(2)} + \theta_{12}^{(2)}a_2^{(2)} + \theta_{13}^{(2)}a_3^{(2)}$, and we transform it into the final output using another activation function τ_2 which approximates our decision rule $a_2^{(3)} = \tau_2(z_1^{(3)}) \approx \varphi(x_1, x_2, x_3; \theta)$, where $\theta \equiv \{\theta_{10}^{(1)}, \theta_{11}^{(1)}, \dots, \theta_{13}^{(2)}\}$ contains all biases and weights. The predicted output is a highly non-linear function of inputs. Hidden layers extract information and condense it in a more abstract way which makes neural-network approximations more flexible, compared to polynomial functions that relate inputs and outputs directly; see Supplement B for a general discussion of neural networks.

⁷ This is a convenient choice since it allows for an easy construction of the derivative function $\sigma'(x) = \sigma(x)(1 - \sigma(x))$. See the online supplement for examples of other common activation functions such as hyperbolic tangent and leaky relu.

Generating data: solving the model where the solution lives. Judd et al. (2011) argue that stochastic simulation methods have a remarkable feature that makes them an ideal candidate for analyzing high dimensional applications: they solve the model only in the area of the state space in which the solution “lives” – the ergodic set. Fig. 2 illustrates this point by showing the ergodic set of a typical representative-agent neoclassical growth model (with two state variables, capital and productivity level).

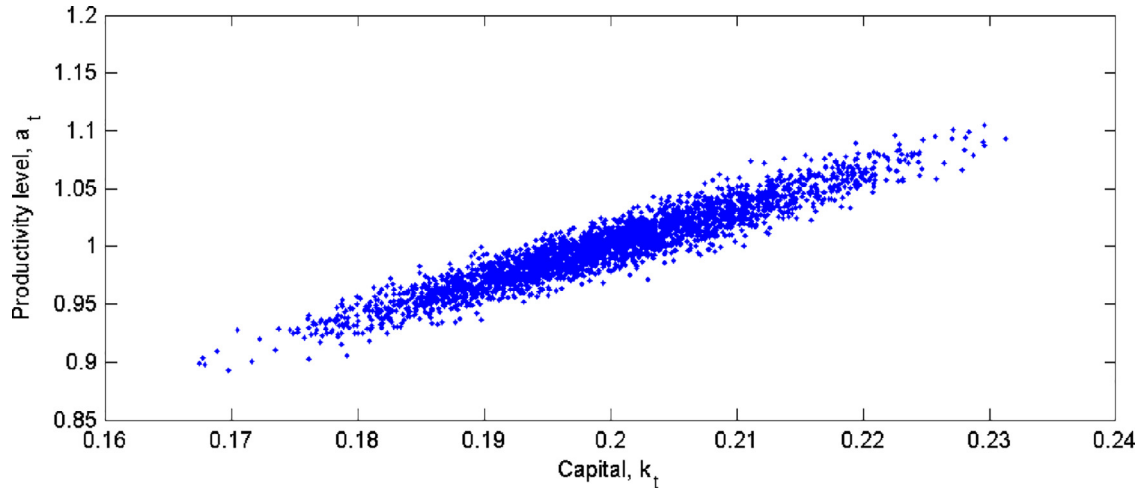


Fig. 2. Ergodic set of a neoclassical growth model.

The ergodic set takes the form of an oval and most of the rectangular area that sits outside of the oval's boundaries is never visited. That means that a solution method that operates on a rectangular domain is wasting computational resources on evaluating points that never occur in equilibrium. In the two-dimensional case, a circle inscribed within a square occupies about 79% of the area of the square, so we save 21% of the total cost. The saving is not big in two-dimensional case but Judd et al. (2011) argues that the ratio $\mathcal{V}^d$ of the volume of a d -dimensional hypersphere to the volume of a d -dimensional hypercube declines rapidly with the dimensionality of the state space

$$\mathcal{V}^d = \begin{cases} \frac{(\pi/2)^{\frac{d-1}{2}}}{1 \cdot 3 \cdot \dots \cdot d} & \text{for } d = 1, 3, 5, \dots \\ \frac{(\pi/2)^{\frac{d}{2}}}{2 \cdot 4 \cdot \dots \cdot d} & \text{for } d = 2, 4, 6, \dots \end{cases} \quad (18)$$

For example, for dimensions three, four, five, ten and thirty, this ratio is 0.52, 0.31, 0.16, $3 \cdot 10^{-3}$ and $2 \cdot 10^{-14}$, respectively. Thus, when focusing on the ergodic set, we face just a tiny fraction of the cost that we would have faced on a fixed hypercube grid. The higher is the dimensionality of a problem, the larger is the reduction in cost.

AiO integration operator: we need just two integration nodes. The AiO operator is a critical technique for solving models with a large number of stochastic shocks. As an illustration, consider the Euler-equation method in the model with ℓ heterogeneous agents who face idiosyncratic shocks $\epsilon_1, \dots, \epsilon_\ell$. In the absence of the AiO operator, the objective function (10) has the form $\Xi(\theta) = E_{(m,s)}(E_{\epsilon_1, \dots, \epsilon_\ell}[f(m, s, \epsilon_1, \dots, \epsilon_\ell; \theta)])^2$, so for each state (m, s) , we need to approximate integral of f across $\epsilon_1, \dots, \epsilon_\ell$. If we consider p integration nodes for each of ℓ idiosyncratic shock, we obtain a tensor product grid with p^ℓ nodes, running into a severe curse of dimensionality. The AiO method addresses the curse of dimensionality in a remarkably simple manner. By using two independent random draws $(\epsilon'_1, \dots, \epsilon'_\ell)$ and $(\epsilon''_1, \dots, \epsilon''_\ell)$, we obtain:

$$E_{(m,s)}(E_{\epsilon_1, \dots, \epsilon_\ell}[f(m, s, \epsilon_1, \dots, \epsilon_\ell)])^2 = E_{(m,s, \epsilon'_1, \dots, \epsilon'_\ell, \epsilon''_1, \dots, \epsilon''_\ell)}[f(m, s, \epsilon'_1, \dots, \epsilon'_\ell)f(m, s, \epsilon''_1, \dots, \epsilon''_\ell)].$$

Independently of the number of shocks in the economy, we need only two random draws (or two batches) for approximating expectation function for each state (m, s) considered. Of course, the AiO approximation of the double integral is crude in any given iteration, but the approximation is unbiased and thus, converges to the true integral over the iterative process. In sum, the AiO method makes numerical integration very cheap.

Stochastic gradient descent method for training: we need just one grid point. Training of multilayer neural network is referred to as *deep learning* because such networks have interconnected topologies with coefficients and weights buried deeply in multiple layers. In data science application, neural networks are typically trained by using variants of a gradient descent method $\theta_{k+1} \leftarrow \theta_k - \lambda_k \nabla \Xi(\theta_k)$, where k is iteration, ∇f is a gradient of f and λ_k is a learning rate. Our definition

of (17) means that expectation and gradient operators are commutable, so that we can approximate the gradient of the integral $\nabla \Xi(\theta) = \nabla E_\omega[\xi(\omega; \theta)]$ with a sample average of the gradient of the integrand $\frac{1}{n} \sum_{i=1}^n \nabla_\theta \xi(\omega_i; \theta_k)$:

$$\theta_{k+1} \leftarrow \theta_k - \lambda_k \left[\frac{1}{n} \sum_{i=1}^n \nabla_\theta \xi(\omega_i; \theta_k) \right]. \quad (19)$$

The limiting case of $n = 1$ in (19) corresponds to a stochastic gradient descent (SGD) method that approximates the gradient of the integral with the gradient of the integrand evaluated in just one randomly selected point, i.e., $\nabla \Xi(\theta_k) = \nabla_\theta E_\omega[\xi(\omega; \theta_k)] \approx \nabla_\theta \xi(\omega_i; \theta_k)$. While stochastic gradient is very imprecise in each given step, it is an unbiased estimate of the true gradient and its cumulative average converges to the true gradient over K iterations $\frac{1}{K} \sum_{k=1}^K \nabla_\theta \xi(\omega_k; \theta_k) \rightarrow \nabla_\theta \Xi(\theta)$, provided that the network parameters converge to their true values $\theta_k \rightarrow \theta$. Thus, an extreme version of our DL method requires just one random grid point for approximation (and just two random nodes for integration). There are other versions of the SGD method, in particular, we will be using a method called ADAM in our numerical analysis; see Supplement C for a review of SGD methods and their convergence properties.

DL solution algorithm. We now combine the above numerical techniques into a DL solution algorithm.

Algorithm 1 has multiple hyperparameters including the topology of neural network, the learning rate, the number of simulation points and integration nodes, and the training method. The algorithm may also include additional hyperparameters.

Algorithm 1 DL algorithm for solving dynamic economic models.

Step 1. Initialize the algorithm:

- i). construct theoretical risk $\Xi(\theta) = E_\omega[\xi(\omega; \theta)]$ (lifetime reward, Euler/Bellman equations);
- ii). define empirical risk $\Xi^n(\theta) = \frac{1}{n} \sum_{i=1}^n \xi(\omega_i; \theta)$;
- iii). define a topology of neural network $\varphi(\cdot, \theta)$;
- iv). fix initial vector of the coefficients θ .

Step 2. Train the machine, i.e., find θ that minimizes the empirical risk $\Xi^n(\theta)$:

- i). simulate the model to produce data $\{\omega_i\}_{i=1}^n$ by using the decision rule $\varphi(\cdot, \theta)$;
 - ii). construct the gradient $\nabla \Xi^n(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla \xi(\omega_i; \theta)$;
 - iii). update the coefficients $\hat{\theta} = \theta - \lambda_k \nabla \Xi^n(\theta)$ and go to step 2.i);
- End Step 2 if the convergence criterion $\|\hat{\theta} - \theta\| < \varepsilon$ is satisfied.

Step 3. Assess the accuracy of constructed approximation $\varphi(\cdot, \theta)$ on a new sample.

ters such as Tykhonov and Lasso regularization parameters for dealing with overfitting and ill-conditioning; see Judd et al. (2011) for a discussion of numerical stability of stochastic simulation methods. Finally, objective function (17) has its own hyperparameters, namely, the relative weights of different model equations. To select the hyperparameters, we use the usual validation procedure of assessing the algorithm performance under different hyperparameter combinations and by selecting those that dominate others in accuracy and speed.

Finally, to implement the DL solution method, we use the Python programming language and Google TensorFlow data platform. Such a platform represents operations on a computational graph, which is automatically optimized. The elements of the graphs, *tensors*, are multidimensional arrays manipulated by efficient vectorized symbolic engines. In particular, gradient computations are performed using automatic differentiation in a numerically stable way, without any user input. Moreover, the graph operations are massively parallelizable on multiple CPU and GPU cores. This is a particularly useful property for our Monte Carlo simulation, in which we evaluate the same function with many draws of shocks for computing conditional expectation functions.

How our deep learning method differs from the conventional projection method. To appreciate the advantages of our DL algorithm, let us recall a canonical projection method of Judd (1992). That method approximates decision rules $\varphi(\cdot; \theta) \approx \varphi$ with a tensor product of Chebyshev polynomial basis functions and constructs a solution on a fixed tensor-product grid of zeros of Chebyshev polynomials. The expectation function $E_\epsilon[\cdot]$ is approximated using Gauss-Hermite quadrature. The algorithm finds θ by minimizing the squared sum of Euler equation residuals $(E_\epsilon[f_j(m, s, x, m', s', x')])^2$ using a Newton-style method. The method is remarkably fast for small problems but becomes increasingly costly as the dimensionality of the problem increases.

What makes the canonical projection method expensive in high dimensional applications? i) The volume of a hypercube domain increases exponentially with the number of state variables; ii) the number of points in tensor-product grid covering that domain grows exponentially with the number of state variables; iii) the volume of a hypercube set for integration across future shocks grows exponentially with the number of shocks; iv) the number of quadrature nodes in the tensor product grid grows exponentially with the number of shocks; v) for each grid point in state space, there are multiple nodes for approximating expectation functions; vi) for more complex models, the cost is higher because we have more equations and more variables to solve for; vii) the number of approximation functions is paired with the number of grid points, so the

number of coefficients in θ grows exponentially as well; viii) the least square approximation suffers from ill-conditioning and numerical instability; ix) the code written by individual researchers is not always optimized for the best performance.

Our DL analysis addresses all these shortcomings: i) our simulation-based domain focuses on the ergodic set in which the solution lives; ii) within that reduced domain, we consider just one or few random grid points on each iteration; iii) we approximate integrals with simulated shocks that again come from the ergodic set; iv) we consider only two random integration nodes on each iteration; v) our AiO operator reduces the cost of integration even further by using a composite random draw for both improving the approximation and evaluating the integral; vi) we collect all the model's equations in one objective function, so that we iterate on all decision rules at once; vii) we use a deep learning neural network, in which we control the topology and number of coefficients; viii) neural networks perform the model reduction and automatically deal with ill conditioning and multicollinearity. ix) we use the state-of-the-art combination of software and hardware that allows for effective GPU parallelization and that leads to remarkable applications in data science. Taken together, these methods will allow us to analyze problems with much larger dimensionality (thousands of state variables) than those studied in the related literature.

4. Numerical analysis of the consumption-saving problem

In this section, we solve a one-agent consumption-saving problem with an occasionally binding borrowing constraint. We show the deep learning method for three different objective function: the lifetime reward, and the Euler- and Bellman-equation residuals. Our experiments are designed to illustrate the role of hyperparameters in the solution, as well as to emphasize some useful features of the proposed method, in particular, its ability to accurately approximate kinks, its capacity to handle ill-conditioned problems and its scalability.

4.1. The consumption-saving problem

We consider a simple consumption-saving problem with a borrowing constraint:

$$\max_{\{c_t, w_{t+1}\}_{t=0}^{\infty}} E_0 \left[\sum_{t=0}^{\infty} \beta^t u(c_t) \right] \quad (20)$$

$$\text{s.t. } w_{t+1} = r(w_t - c_t) + e^{y_t}, \quad (21)$$

$$c_t \leq w_t, \quad (22)$$

where c_t and w_t are consumption and cash-on-hand, respectively; u is a utility function, which is strictly increasing and concave; $\beta \in [0, 1)$ is a discount factor; $r \in (0, \frac{1}{\beta})$ is an interest rate, and initial condition w_0 is given. Exogenous income shock y_t follows an AR(1) process,

$$y_{t+1} = \rho y_t + \sigma \epsilon_t \text{ and } \epsilon_t \sim \mathcal{N}(0, 1), \quad (23)$$

where $|\rho| < 1$ and $\sigma > 0$. The borrowing limit in (22) is set to zero without loss of generality. We parameterize the model by $u(c) = \frac{c^{1-\gamma}-1}{1-\gamma}$ with a risk-aversion coefficient of $\gamma = 2$, and we assume $\beta = 0.9$ and $r = 1.04$.

To facilitate the exposition of the algorithm, we make two simplifying assumptions: First, we assume that the income shock is temporary $y_t = \sigma \epsilon_t$, where $\sigma = 0.1$. Then, we have just one state variable w , which is convenient for illustrating decision functions on two-dimensional plots. Second, we assume that y is drawn from its ergodic (Normal) distribution, but w is drawn from a uniform distribution in an interval $[w_1, w_2]$. That allows us to abstract from the convergence of simulated series for w and to concentrate on the convergence of the coefficients θ . Later in the paper, we consider models with a large number of state variables, and we draw both endogenous and exogenous state variables from their ergodic distributions.

Euler equation. The solution to (20)–(23) can be characterized by Kuhn-Tucker conditions

$$A \geq 0, \quad H \geq 0 \text{ and } AH = 0, \quad (24)$$

where $A = w - c$ and $H \equiv u'(c) - \beta r E_\epsilon [u'(c')]$ is a Lagrange multiplier. To construct a DL objective function, we rewrite Kuhn Tucker conditions that have inequality constraints (24) as the Fischer-Burmeister (FB) function that holds with equality

$$\Psi^{FB}(a, h) = a + h - \sqrt{a^2 + h^2} = 0, \quad (25)$$

where $a \equiv 1 - \frac{c}{w}$ and $h \equiv 1 - \frac{\beta r E_\epsilon [u'(c')]}{u'(c)}$ are expressed in unit-free form. The FB function is similar to the minimum function $\min\{a, h\} = 0$ and leads to the same solution (24) as the Kuhn Tucker conditions but it is differentiable; see, e.g., Jiang (1996) for a discussion. Even though the two terms a and h are in unit free form, it might be necessary to add a weight

ν that reflects the relative importance of the two objectives a and h , i.e., in general, we may need to consider $\Psi^{FB}(a, \nu h)$, where $\nu \in (0, \infty)$.

Bellman equation. The solution to (20)–(23) can be also characterized by Bellman equation:

$$V(y, w) = \max_{c, w'} \{u(c) + \beta E_\epsilon [V(y', w')]\}, \quad (26)$$

subject to constraints (21) and (22) and transition equation (23) written in recursive form. The maximum operator of the Bellman equation is also characterized by the Kuhn-Tucker conditions (24) however the Lagrange multiplier is defined here in terms of the derivative of the value function $H \equiv u'(c) - \beta E_\epsilon \left[\frac{\partial V(y', w')}{\partial w'} \right]$. Consequently, we can get rid of the inequality constraints in the Kuhn-Tucker conditions by using the same FB function (25) as we did for the Euler-equation method. The first term of the FB function is the same, i.e. $a \equiv 1 - \frac{c}{w}$ but the second term will be defined in terms of the derivative of the value function $h \equiv 1 - \frac{\beta E_\epsilon \left[\frac{\partial V(y', w')}{\partial w'} \right]}{u'(c)}$.

4.2. Deep learning solution method

To implement the lifetime reward, Euler and Bellman methods, we used Algorithm 1 formulated in Section 2. The only difference between three methods consists in how we simulate the model in Step 2i). We will describe this step separately for each of the method considered.

We construct the solutions using a neural network with two identical hidden layers, composed of (leaky) relu (rectified linear units) neurons. We compare the results under four neural networks with 8×8 , 16×16 , 32×32 and 64×64 neurons in two hidden layers, respectively. We parameterize consumption to wealth ratio $\frac{c_t}{w_t}$, unit-free Lagrange multiplier h_t and value function V_t :

$$\begin{aligned} \frac{c_t}{w_t} &= \sigma(\zeta_0 + \eta(y_t, w_t; \vartheta)) \equiv \varphi(y_t, w_t; \theta), \\ h_t &= \exp(\zeta_0 + \eta(y_t, w_t; \vartheta)) \equiv h(y_t, w_t; \theta), \\ V_t &= \zeta_0 + \eta(y_t, w_t; \vartheta) \equiv V(y_t, w_t; \theta), \end{aligned}$$

where $\eta(\cdot; \vartheta)$ is a neural network, $\theta \equiv (\zeta_0, \vartheta)$ and $\sigma(x) = \frac{1}{1+e^{-x}}$ is a sigmoid transformation. Our lifetime reward method will use only $\varphi(y_t, w_t; \theta)$, the Euler method will use both $\varphi(y_t, w_t; \theta)$ and $h(y_t, w_t; \theta)$ and finally, the Bellman method will use all three of them.

Our parametrization is constructed to take into account the properties of economic variables. A sigmoid transformation used for $\frac{c_t}{w_t}$ ensures that the consumption share in wealth $\varphi(\cdot; \theta)$ is bounded to be in an interval $[0, 1]$ and exponentiation used for h_t ensures that it is always nonnegative; and finally, for value function we used a linear activation, i.e., we place no restrictions on $V(\cdot; \theta)$ range. We initialize at $\zeta_0 = 0$ for all parametrized functions. The remaining parameters ϑ are initialized randomly: we used the “he” and “glorot” uniform distributions for the biases and weights, respectively.

We train the model using a version of the stochastic gradient descent method, called ADAM, with an overall learning rate of $\lambda = 0.001$ (in that method, the overall learning rate is optimally adjusted for each coefficient; see Appendix B for details). We perform training over $K = 50,000$ epochs (iterations); and in each epoch, we draw 64 random grid points by using the interval for wealth $[w_1, w_2] = [0.1, 4]$. To evaluate the accuracy, we produce 8,192 random draws and use the constructed decision rules to produce the lifetime reward and unit-free Euler equation residuals. To approximate integrals in the accuracy test, we use an accurate 10-node Gauss-Hermite quadrature rule. We wrote the code in Python using Google TensorFlow platform version 1.14.0, and we use a laptop with Intel(R) Core(TM) i7-7500U (2.70 GHz), RAM 16GB with 4 physical (and 8 virtual) cores.

4.3. Lifetime reward

The objective function for the lifetime reward for the model (20)–(23) follows directly from our general exposition (8). For a random draw $\omega = (y_0, w_0, \epsilon_1, \dots, \epsilon_T)$ and time horizon T , the objective function associated with the decision rule $\frac{c_t}{w_t} = \varphi(y_t, w_t; \theta)$ is given by

$$\Xi(\theta) = E_\omega[\xi(\omega; \theta)] \equiv E_{(y_0, w_0, \epsilon_1, \dots, \epsilon_T)} \left[\sum_{t=0}^T \beta^t u(c_t) \right]. \quad (27)$$

To construct (27), i.e., to implement Step 2i) of Algorithm 1, we use the assumed decision rule $\frac{c_t}{w_t} = \varphi(y_t, w_t; \theta)$ and simulate the model forward together with transitions (21)–(23).

Fig. 3 displays the outcome of training of neural network on the objective function (27) (on the horizontal axis, $K = 50,000$ epochs appears as $4 \cdot \log_{10} 5$).

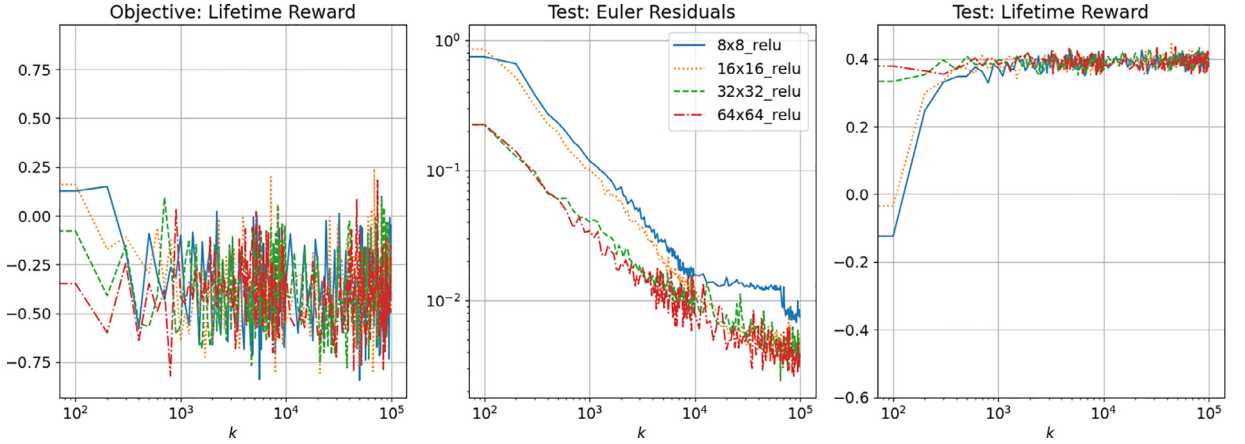


Fig. 3. Lifetime reward in the consumption-saving model.

In the first panel of the figure, we illustrate how the objective function – lifetime reward – changes over the epochs; in the second panel, we show the size of the Euler equation residuals constructed on 8,192 test points using the FB function (25) and accurate quadrature integration, and finally, in the last panel, we plot the evolution of lifetime reward. As is seen from the second panel, the Euler residuals are smaller for a neural network with a larger number of neurons (the smallest residuals are for the one with 64×64 neurons in two hidden layers). They are of order $10^{-2.75}$ (i.e., 0.18%), which is fairly low, given that we solve the model with a kink in the decision rule. In terms of the lifetime reward, all the solutions gradually converge to the same level.

4.4. Euler-equation method with Kuhn-Tucker conditions

For our model with borrowing constraint, the Euler-residual (10) objective function is given by the Fischer-Burmeister function (25):

$$E_{(y,w)} \left[\Psi^{FB} \left(1 - \frac{c}{w}, 1 - \frac{\beta r E_{\epsilon} [u'(c')]}{u'(c)} \right) \right]^2. \quad (28)$$

In Section 1.3, we constructed an AiO expectation operator that makes it possible to combine the two expectation operators $E_{(y,w)}[\cdot]$ and $E_{\epsilon}[\cdot]$ in one by approximating $E_{\epsilon}[\cdot]$ using two uncorrelated shocks (11). However, the Fischer-Burmeister function has expectation term $E_{\epsilon} [u'(c')]$ inside of the square root, in addition to the squared residual. To extend the AiO operator for the FB function, we introduce a separate approximation for the Lagrange multiplier h and rewrite (28) as a composite objective

$$E_{(z,w)} \left[\Psi^{FB} \left(1 - \frac{c}{w}, 1 - h \right) \right]^2 + v_h \left[\frac{\beta r E_{\epsilon} [u'(c')]}{u'(c)} - h \right]^2, \quad (29)$$

where v_h is an exogenous weight. Using the technique of two uncorrelated shocks (11), we then arrive to an objective function with the AiO operator that combines integration with respect to z , w and ϵ :

$$\Xi(\theta) = E_{\omega} [\xi(\omega; \theta)] = E_{(z,w,\epsilon_1,\epsilon_2)} \left\{ \left[\Psi^{FB} \left(1 - \frac{c}{w}, 1 - h \right) \right]^2 + v_h \left[\frac{\beta r u'(c')|_{\epsilon=\epsilon_1}}{u'(c)} - h \right] \left[\frac{\beta r u'(c')|_{\epsilon=\epsilon_2}}{u'(c)} - h \right] \right\}. \quad (30)$$

The objective function (30) is the one that we minimize by deep learning Algorithm 1. To construct the objective function $\Xi(\theta)$ in Step 2i) of Algorithm 1, we produce a random draw $\omega = (y, w, \epsilon_1, \epsilon_2)$ and use the decision rules for consumption share $\frac{c}{w} = \varphi(y, w; \theta)$ and Lagrange multiplier $h = h(y, w; \theta)$ and transition equations (21) and (23).

The results of training under the objective (30) are shown in Fig. 4.

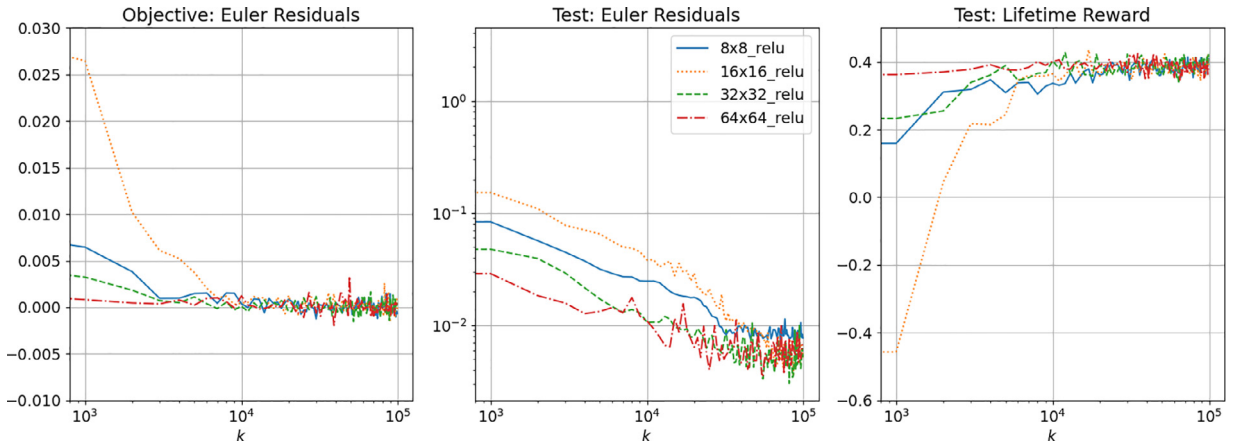


Fig. 4. Euler-equation method in the consumption-saving model.

Again, as in the previous figure, we plot the objective function – Euler residuals, the size of the Euler equation residuals on the test data, and the evolution of the lifetime reward in the first, second and third panels, respectively. As is evident from the second panel, the Euler-equation method is slightly more accurate than the lifetime reward method. We again observe that the deep learning method with a larger number of neurons is more accurate.

4.5. Objective 3: Bellman equation

For the Bellman-equation method, the objective function (14) includes the residuals in the Bellman equation and the residuals in the Fischer-Burmeister function that characterize the maximum operator in the Bellman equation

$$E_{(z,w)} \left[V(z, w; \theta_1) - u(c) - \beta E_{\epsilon} [V(z', w'; \theta_1)] \right]^2 + v E_{(y,w)} \left[\Psi^{FB} \left(1 - \frac{c}{w}, 1 - \frac{\beta E_{\epsilon} \left[\frac{\partial}{\partial w'} V(z', w'; \theta) \right]}{u'(c)} \right) \right]^2. \quad (31)$$

By following the same approach as in the case of the Euler-equation method, we introduce a separate approximation for the Lagrange multiplier h and apply the method of two uncorrelated shocks to obtain the following objective function for the Bellman-equation method:

$$\begin{aligned} \Xi(\theta) = E_{\omega} [\xi(\omega; \theta)] &\equiv E_{(y,w,\epsilon_1,\epsilon_2)} \left\{ \left[V(y, w; \theta) - u(c) - \beta V(y', w'; \theta) \right]_{\epsilon=\epsilon_1} \right. \\ &\times \left[V(y, w; \theta) - u(c) - \beta V(y', w'; \theta) \right]_{\epsilon=\epsilon_2} + v \left[\Psi^{FB} \left(1 - \frac{c}{w}, 1 - h \right) \right]^2 \\ &\left. + v_h \left[\frac{\beta \frac{\partial}{\partial w'} V(y', w'; \theta)_{\epsilon=\epsilon_1}}{u'(c)} - h \right] \left[\frac{\beta \frac{\partial}{\partial w'} V(y', w'; \theta)_{\epsilon=\epsilon_2}}{u'(c)} - h \right] \right\}. \end{aligned} \quad (32)$$

The objective function (32) is the one that we use as an input for deep learning Algorithm 1. To implement Step 2i), i.e. to construct the objective function (32), we produce a random draw $\omega = (y, w, \epsilon_1, \epsilon_2)$, use the decision rules for value function $V(y, w; \theta)$, consumption share $\frac{c}{w} = \varphi(y, w; \theta)$ and Lagrange multiplier $h = h(y, w; \theta)$ and transition equations (21) and (23).

The choice of weights v and v_h is a nontrivial problem. While in the previous methods, we expressed the residuals in unit free form, so that we could use unit weights, it is not a feasible approach here. Value function for our model has the range from negative to positive numbers. If we do it unit free, we will get sometimes a division by numbers close to zero. Additionally, the residuals can switch from positive to negative leading to spurious iterations of the gradient algorithm. In our application, we choose weights so that the size of the residuals was approximately the same in all three equations.

Fig. 5 plots the training process for Objective 3 when neural networks have different number of neurons.

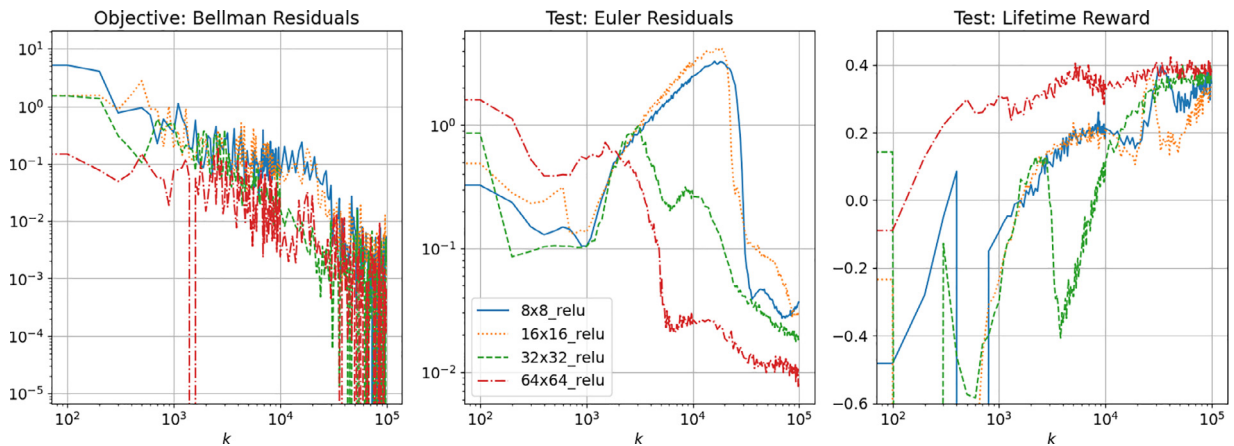


Fig. 5. Bellman-equation method in the consumption-saving model.

Fig. 5 displays the changes in the objective function over the epochs (the first panel), the size of test Euler-equation residuals (the second panel), and the size of the lifetime reward (the third panel). The value iterative method is less accurate than the two previous methods. This fact is not surprising: Coleman et al. (2018) compare the conventional VFI method which solves the Bellman equation by constructing the decision function via direct maximization, as we do here, versus otherwise identical methods that use derivatives of the value function via first-order and envelope conditions. They find that the methods that approximate the derivatives of V are far more accurate than the methods that approximate V alone. In the second panel, Euler residuals are higher for this objective function than for the other two, although they decrease to 10^{-2} (i.e., 1%) when the number of neurons in two hidden layers increases to 64×64 .

4.6. Comparing decision rules

In Fig. 6, we plot decision rules for the three methods and simulated series for wealth.

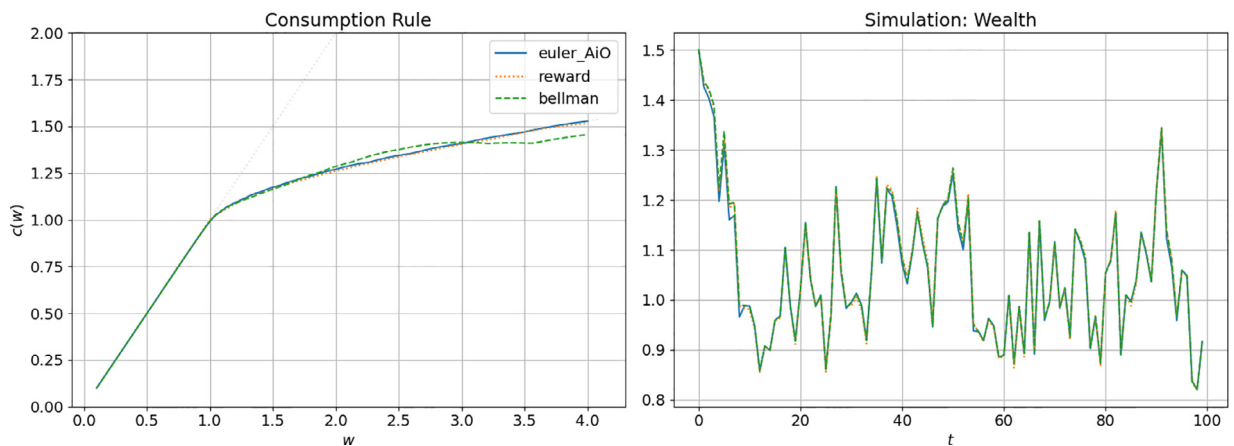


Fig. 6. Comparison of methods in the consumption-saving model.

We observe that the decision rule for the Bellman method appears to be less accurate than the other two methods, however, simulated series are remarkably close to one another across the three methods. What is surprising that the three methods visually coincide in the area of kink. The second panel shows that there is a substantial variability in wealth (cash-on-hand) over a simulation of 100 periods.

4.7. Euler-equation method: Sensitivity results

We now limit attention to the Euler-equation method and dummyTXdummy– consider how its performance is affected by specific techniques used. In Fig. 7, we plot the convergence of the objective function.

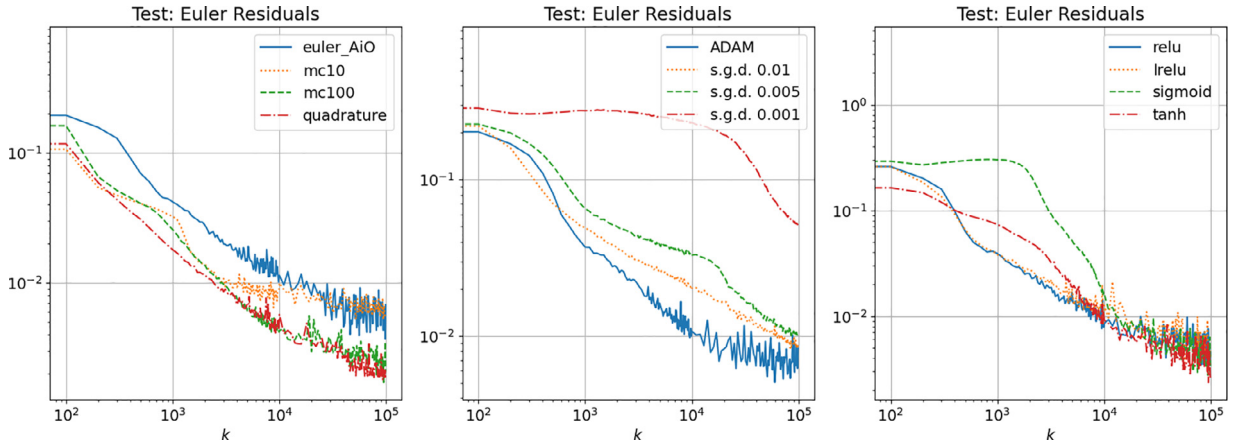


Fig. 7. Comparison of integration, SGD and activation methods.

In the first panel, we show the results in which we vary an integration method, namely, we consider the AiO method with 2 nodes, Monte Carlo (MC) method with 10 random draws, MC method with 100 random draws and very accurate Gauss Hermite quadrature with 10 nodes. We observe that the integration method we use plays a visible role in convergence. A very accurate quadrature method leads to the fastest convergence while low accuracy MC leads to slower convergence than the AiO method. In the second panel, we consider four training methods, namely, SGD with an updating parameter $\lambda \in \{0.01, 0.005, 0.001\}$ and ADAM. We see that ADAM leads to the fastest convergence even though the convergence is noisier at the end with ADAM than with the other methods. Finally, in the last panel, we compare the convergence with different activation functions, namely, relu, lrelu, sigmoid and tanh, and we document that they lead to comparable results.

In Fig. 8, we illustrate the scalability of the Euler-equation method. TensorFlow has a possibility of using batches where the same estimation is ran in parallel on multiple virtual cores. We vary the batch size from $N = 8$ to $N = 8,192$ draws and we document running time and Euler-equation residuals.

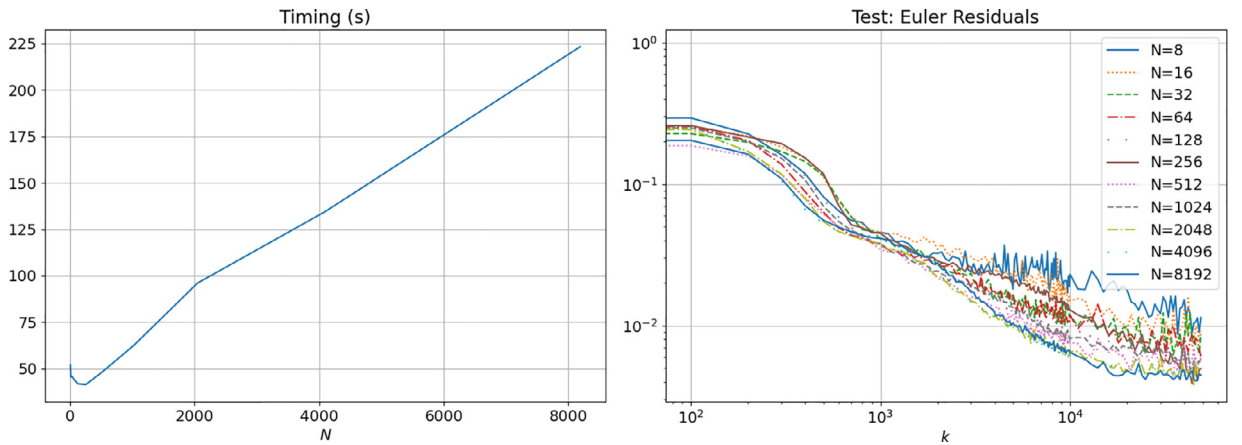


Fig. 8. Effects of the batch size on the accuracy and speed.

In the left panel, we see that the training time changes roughly linearly with the batch size. In the right panel, we observe that a larger batch size leads to more accurate solutions although the convergence rate is similar for all batch sizes. The same is true for all the methods built on the AiO operator.

4.8. Multicollinearity, ill-conditioning and model reduction

We consider a version of the consumption-saving problem with multiple shocks,

$$\max_{\{c_t, w_{t+1}\}_{t=0}^{\infty}} E_0 \left[\sum_{t=0}^{\infty} \beta^t e^{\chi_t} u(c_t) \right] \quad (33)$$

$$\text{s.t. } w_{t+1} = re^{Q_t}(w_t - c_t) + e^{y_t}e^{p_t}, \quad (34)$$

$$c_t \leq w_t, \quad (35)$$

where $\{y_t, p_t, Q_t, \chi_t\} \equiv z_t$ is a vector of exogenous state variables, which includes a temporary income shock y_t , a long-lasting income shock p_t , an interest-rate shock Q_t and a preference shock χ_t . We assume that each exogenous state variable $z_j \in \{y, p, Q, \chi\}$ follows an AR(1) process,

$$z_{j,t+1} = \rho_j z_{j,t} + \sigma_j \epsilon_{j,t} \text{ and } \epsilon_{j,t} \sim \mathcal{N}(0, 1), \quad (36)$$

where $|\rho_j| < 1$ and $\sigma_j > 0$. We parameterize the model by $u(c) = \frac{c^{1-\gamma}-1}{1-\gamma}$ with a risk-aversion coefficient of $\gamma = 2$, and we assume $\beta = 0.9$, $r = 1.04$, $\rho_y = 0.9$ and $\sigma_y = 0.1$; $\rho_p = 0.999$ and $\sigma_p = 0.001$; $\rho_Q = 0.2$ and $\sigma_Q = 0.001$; and $\rho_\chi = 0.9$ and $\sigma_\chi = 0.01$.

The lifetime-reward objective function (27) is built on the AiO operator and it is directly suitable for high-dimensional applications. In Fig. 9, we present the results obtained with reward maximization for the multistate model (20)–(23). We use 64 relu nodes in each of the two hidden layers, and the training method was ADAM.

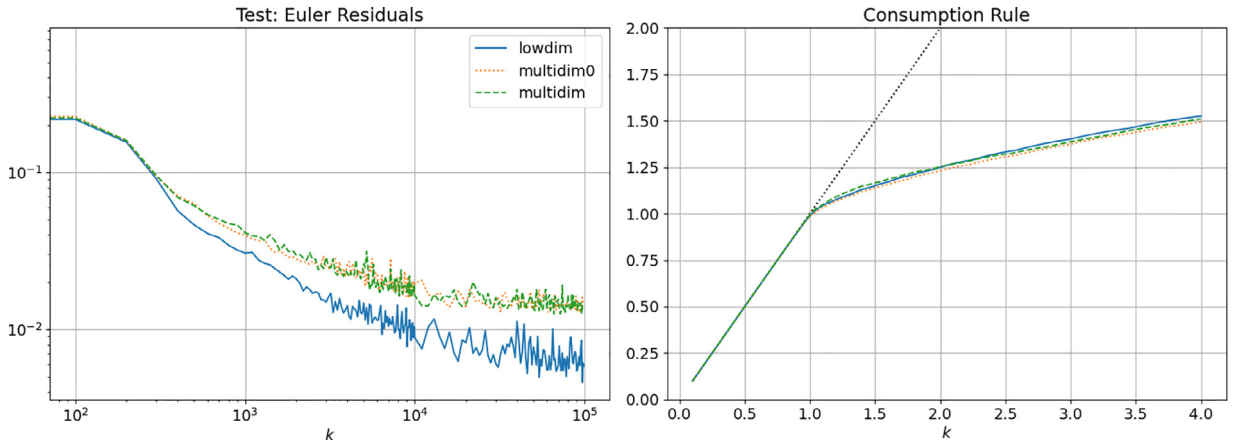


Fig. 9. Multicollinearity and model reduction in the multishock model.

Three cases are considered in the figure. The main case is the multi-shock model denoted by "multidim". The other two cases, "lowdim" and "multidim0", correspond to a version of the model with only one income shock. But the two models differ in the inputs that we supply to the neural network: in the "lowdim" model, the irrelevant shocks other than income shock are not supplied at all, while in "multidim0", they are supplied to the neural network by setting all of them equal to zero. Thus, the latter model has perfect multicollinearity, so that the inverse problem is ill-conditioned and cannot be solved with conventional regression or approximation methods, such as ordinary-least squares (OLS).

There are two main results to learn from this experiment: First, neural-network approximations do not suffer from multicollinearity and ill conditioning, unlike the conventional polynomial approximation. Training of the model with zero shocks leads to the same solution and has roughly the same convergence rate as those of the other two models. This experiment illustrates how neural networks can do the model reduction: they learn to ignore the effect of nonexistent shocks although at some additional initial cost (i.e., the residuals of the last model are slightly larger in the beginning of training than those of the other models). Second, training in the multi-shock model has approximately the same convergence rate as that of the other models. The cost of iteration in the multi-shock model is slightly larger than in the unishock model but this difference is relatively small. This finding indicates that the proposed solution method is potentially tractable in problems with high dimensionality.

5. Numerical analysis of Krusell and Smith's (1998) model

We now use our deep learning method to solve Krusell and Smith (1998) model. We formulate the model in terms of cash-in-hand to make it comparable to the consumption-saving problem studied in Section 4.

5.1. Krusell and Smith (1998) model

The economy consists of a set of heterogeneous agents $i = 1, \dots, \ell$ that are identical in fundamentals but differ in productivity and capital. Each agent i solves

$$\max_{\{c_t^i, k_{t+1}^i\}_{t=0}^{\infty}} E_0 \left[\sum_{t=0}^{\infty} \beta^t u(c_t^i) \right] \quad (37)$$

$$\text{s.t. } w_{t+1}^i = R_{t+1}(w_t^i - c_t^i) + W_{t+1} \exp(y_{t+1}^i), \quad (38)$$

$$c_t^i \leq w_t^i, \quad (39)$$

where c_t^i , w_t^i , y_t^i , R_t , W_t and $k_{t+1}^i = w_t^i - c_t^i$ are consumption, cash-on-hand, labor productivity, interest rate, wage and next-period capital, respectively. Initial condition (y_0^i, w_0^i) is given. The individual productivity evolves as

$$y_{t+1}^i = \rho_y y_t^i + \sigma_y \epsilon_t^i \text{ with } \epsilon_t^i \sim \mathcal{N}(0, 1). \quad (40)$$

The production side of the economy is described by a Cobb-Douglas production function $z_t k_t^\alpha$, where $\alpha \in (0, 1)$ and z_t is an aggregate productivity shock,

$$z_{t+1} = \rho_z z_t + \sigma_z \epsilon_t \text{ with } \epsilon_t \sim \mathcal{N}(0, 1). \quad (41)$$

Initial condition z_0 is given. The equilibrium prices are

$$R_t = 1 - d + z_t \alpha k_t^{\alpha-1} \left[\sum_{i=1}^{\ell} \exp(y_t^i) \right] \text{ and } W_t = z_t (1 - \alpha) k_t^\alpha \left[\sum_{i=1}^{\ell} \exp(y_t^i) \right], \quad (42)$$

where $k_t = \sum_{i=1}^{\ell} k_t^i$ is aggregate capital, and $d \in (0, 1]$ is the depreciation rate. Note that (38) implies that $w_t^i = R_t k_t^i + W_t \exp(y_t^i)$. We parametrize the model by $u(c) = \frac{c^{1-\gamma}-1}{1-\gamma}$ with a risk-aversion coefficient of $\gamma = 1$ and assume $\beta = 0.96$, $\rho = 0.95$, $\sigma = 0.01$, $\rho_y = 0.9$, and $\sigma_y = 0.2(1 - \rho_y^2)^{1/2}$.

5.2. Deep learning solution algorithm

Our analysis of Krusell and Smith (1998) model parallels that of the consumption-saving problem of Section 4. We again construct the solution using the lifetime reward, Euler and Bellman objectives.

State space. The state space consists of the state variables of all agents $\{y_t^i, w_t^i\}_{i=1}^{\ell}$, as well as the aggregate shock z_t . Since agents are homogeneous in fundamentals, as in Krusell and Smith (1998), we need just one $2\ell + 1$ -dimensional decision and value functions to characterize the choices of all ℓ agents.⁸ If agents were heterogenous in fundamentals, we would need to construct separate decision and value functions for each heterogenous agent; each of such functions has $2\ell + 1$ dimensions.

Parameterization. Like in the consumption-saving problem, we parametrize the consumption to wealth ratio $\frac{c_t^i}{w_t^i}$, unit-free Lagrange multiplier h_t^i and value function V_t^i :

$$\begin{aligned} \frac{c_t^i}{w_t^i} &= \sigma(\zeta_0 + \eta(y_t^i, w_t^i, D_t, z_t; \vartheta)) \equiv \varphi(\cdot; \theta), \\ h_t^i &= \exp(\zeta_0 + \eta(y_t^i, w_t^i, D_t, z_t; \vartheta)) \equiv h(\cdot; \theta), \\ V_t^i &= \zeta_0 + \eta(y_t^i, w_t^i, D_t, z_t; \vartheta) \equiv V(\cdot; \theta), \end{aligned}$$

where $\eta(\cdot; \vartheta)$ is a neural network, $D_t \equiv \{y_t^i, w_t^i\}_{i=1}^{\ell}$ is the distribution, $\theta \equiv (\zeta_0, \vartheta)$ and $\sigma(x) = \frac{1}{1+e^{-x}}$.⁹

A sigmoid transformation of $\varphi(\cdot; \theta)$ ensures that $\frac{c_t^i}{w_t^i}$ is in the interval $[0, 1]$; the exponentiation of h_t ensures that it is nonnegative; there is no restriction on the range of $V(\cdot; \theta)$. The parameter ζ_0 is calibrated, and the biases and weights are initialized randomly by using "he" and "glorot" uniform distributions, respectively. In the baseline case, we use a neural network with a sigmoid activation function and two hidden layers of 64×64 neurons.

Simulation. Our implementation of the deep learning solution method follows Algorithm 1 we used for the consumption-saving problem. The algorithm is remarkably straightforward: we simulate a panel of heterogeneous agents forward and train their decision functions as we go. For the given economy's state $(\{w_t^i, y_t^i\}_{i=1}^{\ell}, z)$ and neural network coefficients $\theta \equiv (\zeta_0, \vartheta)$, we do the following calculations to implement the simulation step 2i):

1. Compute $\frac{c_t^i}{w_t^i} = \varphi(y_t^i, w_t^i, D_t, z_t; \theta)$ and $k_{t+1}^i = w_t^i - c_t^i$ for each agent $i = 1, \dots, \ell$.
2. Draw y_{t+1}^i for $i = 1, \dots, \ell$ and z_{t+1} using (40) and (41), respectively.

⁸ Since the true decision and value functions are invariant to any permutation of $\{y_t^i, w_t^i\}_{i=1}^{\ell}$, neural networks should eventually learn this symmetry. In general, a faster learning speed could be achieved if the symmetry is imposed in the solution method. For instance, because of unequal initial asset levels, some agents are given higher weight in the objective functions than the others. By reshuffling randomly the positions of agents, we can prevent overfitting during the training.

⁹ We do not use a recursive representation for Krusell and Smith (1998) model but keep the time subscripts which is more appropriate for describing a solution constructed on stochastic simulation.

3. Compute prices R_{t+1} and W_{t+1} from (42) given $k_{t+1} = \sum_{i=1}^{\ell} k_{t+1}^i$.
4. Compute next period cash-in-hand $w_{t+1}^i = R_{t+1}k_{t+1}^i + W_{t+1} \exp(y_{t+1}^i)$.
5. Compute $\frac{c_{t+1}^i}{w_{t+1}^i} = \varphi(y_{t+1}^i, w_{t+1}^i, D_{t+1}, z_{t+1}; \theta)$ for $i = 1, \dots, \ell$.
6. Evaluate the objective function (lifetime reward, Euler and Bellman residuals), train the neural networks and go to next iteration.

We simulate the model over $K = 300,000$ periods, however, we perform the training only each 10th period.¹⁰ In each iteration, we use 100 simulated points in each iteration. We use ADAM with the learning rate of $\lambda = 0.001$. As the machine is trained and the panel is simulated, the decision functions are refined jointly with the ergodic distribution.

Perfect multicollinearity. To parameterize the decision and value functions, we represent the state space as $(y_t^i, w_t^i, \{y_t^i, w_t^i\}_{i=1}^{\ell}, z_t)$, i.e., we list the individual variables twice, as the state variables of a agent i and as a part of the distribution $\{y_t^i, w_t^i\}_{i=1}^{\ell}$. A repetition implies perfect multicollinearity in explanatory variables, so that the inverse problem is not well defined. Such a multicollinearity would break down a conventional numerical method which solves the inverse problem but neural networks can learn to ignore redundant colinear variables, as we saw earlier. Even though it is possible to design a transformation that avoids a repetition of variables, it would require cumbersome permutations. We find it easier to keep the repeated variables.

Model reduction. We solve the models with $\ell = 1,000$ agents which corresponds to $2\ell + 1 = 2,001$ state variables. How can the DL method deal with such a huge state space? In addition to focusing on the ergodic set, cheap AiO integration and stochastic optimization, we invoke the remarkable property of neural networks to perform model reduction. When we supply a large number of state variables to the input layer, the neural network condenses the information into 64 neurons in two hidden layers, making it more abstract and compact. In a sense, this procedure is similar to the photo compression or principal component transformation when a large set of variables is condensed into a smaller set of principal components without losing essential information; see Goodfellow et al. (2016) for a discussion of neural networks.

5.3. Lifetime reward

The objective function for the lifetime reward for the model (37)–(41) follows directly from our general exposition (8):

$$\Xi(\theta) = E_{\omega}[\xi(\omega; \theta)] \equiv E_{(Y_0, W_0, Z_0, \Sigma, \epsilon)} \left[\sum_{t=0}^T \beta^t u(c_t) \right], \quad (43)$$

where the transitions are determined by (38)–(41); $Y_0 = (y_0^1, \dots, y_0^{\ell})$, $W_0 = (w_0^1, \dots, w_0^{\ell})$ and z are the economy's state produced over stochastic simulation; $\Sigma \equiv (\epsilon_1^1, \dots, \epsilon_1^{\ell}, \dots, \epsilon_T^1, \dots, \epsilon_T^{\ell})$ represents shocks to productivity of all heterogeneous agents $i = 1, \dots, \ell$ over the periods $t = 1, \dots, T$; and $\epsilon = (\epsilon_1, \dots, \epsilon_T)$ is the sequence of innovations to aggregate productivity.

There is an important conceptual question on how to train the objective function (43). We are solving for competitive equilibrium so we must maximize the utility of each agent with respect to her own variables but not with respect to variables of other agents. In practice, we achieve this by "muting" in TensorFlow the gradient of the objective function of a given agent with respect to variables of the other agents.

Fig. 10 displays the outcome of training of neural network on the objective function (43).

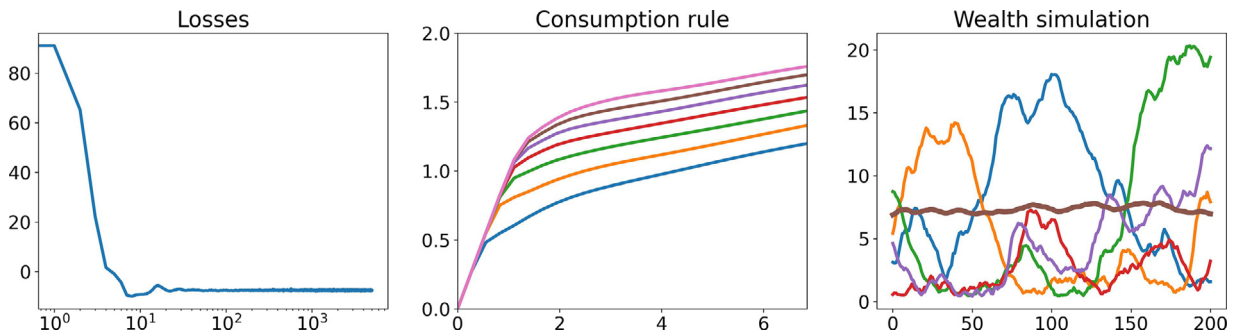


Fig. 10. Lifetime reward in Krusell and Smith (1998) model.

¹⁰ Since random variables are autocorrelated in our model, the stochastic gradient is correlated over time and hence, it is biased. To reduce the bias, we train the model on cross-sections which are sufficiently separated in time instead of using all consecutive periods.

In the first panel, we illustrate how the loss function changes during training; in second panel, we show the consumption functions depending on individual wealth under 7 different productivity levels ranging from -2 to 2 of the standard deviations of productivity (to produce these decision rules, we set the aggregate state and productivity of all other agents to their steady state levels). Finally, in the last panel, we show a simulated wealth series for 5 heterogeneous agents selected randomly from the sample. We see that the consumption decision function is similar to the one shown in Fig. 3 for the one-consumer problem. We also observe that the simulated series for wealth are stationary; they fluctuate within a reasonable range, occasionally reaching the borrowing limit.

5.4. Euler-equation method with Kuhn-Tucker conditions

The Euler objective function for [Krusell and Smith \(1998\)](#) model is parallel to the objective (30) of the consumption-saving problem. Using the technique of two uncorrelated shocks (11) to facilitate the AiO operator, we obtain:

$$\begin{aligned} \Xi(\theta) = E_{\omega}[\xi(\omega; \theta)] = E_{(Y_t, W_t, Z_t, \Sigma_1, \Sigma_2, \epsilon_1, \epsilon_2)} \left\{ \left[\Psi^{FB} \left(1 - \frac{c_t^i}{w_t^i}, 1 - h_t^i \right) \right]^2 \right. \\ \left. + v \left[\frac{\beta R_{t+1} u'(c_{t+1}^i) |_{\Sigma=\Sigma_1, \epsilon=\epsilon_1}}{u'(c_t^i)} - h_t^i \right] \left[\frac{\beta R_{t+1} u'(c_{t+1}^i) |_{\Sigma=\Sigma_2, \epsilon=\epsilon_2}}{u'(c_t^i)} - h_t^i \right] \right\}. \end{aligned} \quad (44)$$

where the transitions are determined by (38)–(41); $Y_t = (y_t^1, \dots, y_t^\ell)$ and $W_t = (w_t^1, \dots, w_t^\ell)$ and z_t are the economy's state produced stochastic simulation; $\Sigma_1 = (\epsilon_1^1, \dots, \epsilon_1^\ell)$, $\Sigma_2 = (\epsilon_2^1, \dots, \epsilon_2^\ell)$ are two uncorrelated random draws of individual productivity shocks; and ϵ_1, ϵ_2 are two uncorrelated random draws for the aggregate productivity innovations.

The results of training under the objective (44) are shown in Fig. 11.

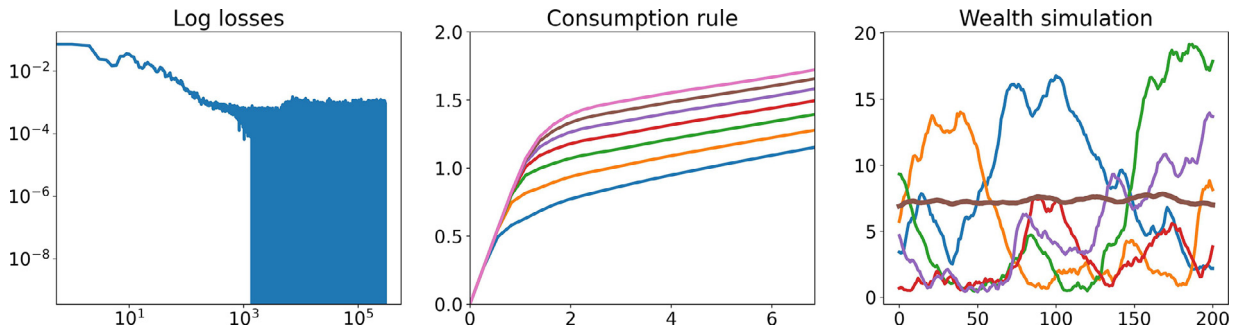


Fig. 11. Euler-equation method in Krusell and Smith's (1998) model.

For the Euler-equation method, the residuals are generally below 10^{-3} (i.e., a fraction of a percentage point). The solutions are very similar to those produced by the lifetime reward maximization method and presented in the previous section.

5.5. Objective 3: Bellman equation

The Bellman objective function for [Krusell and Smith \(1998\)](#) model is also parallel to the objective function (32) derived for the consumption-saving problem. In particular, using the technique of two uncorrelated shocks (11), we then arrive to an objective function with the AiO operator:

$$\begin{aligned} \Xi(\theta) = E_{\omega}[\xi(\omega; \theta)] \equiv E_{(Y_t, W_t, Z_t, \Sigma_1, \Sigma_2, \epsilon_1, \epsilon_2)} \left\{ \left[V(s_t^i; \theta) - u(c_t^i) - \beta V(s_{t+1}^i; \theta) \right]_{\Sigma=\Sigma_1, \epsilon=\epsilon_1} \right. \\ \times \left[V(s_t^i; \theta) - u(c_t^i) - \beta V(s_{t+1}^i; \theta) \right]_{\Sigma=\Sigma_2, \epsilon=\epsilon_2} + v \left[\Psi^{FB} \left(1 - \frac{c_t^i}{w_t^i}, 1 - h_t^i \right) \right]^2 \\ \left. + v_h \left[\frac{\beta \frac{\partial}{\partial w_{t+1}^i} V(s_{t+1}^i; \theta) |_{\Sigma=\Sigma_1, \epsilon=\epsilon_1}}{u'(c_t^i)} - h_t^i \right] \left[\frac{\beta \frac{\partial}{\partial w_{t+1}^i} V(s_{t+1}^i; \theta) |_{\Sigma=\Sigma_2, \epsilon=\epsilon_2}}{u'(c_t^i)} - h_t^i \right] \right\}, \end{aligned} \quad (45)$$

where s_t^i is a vector of the state variables and all other variables are defined as in (44).

The results of training under the objective (45) are shown in Fig. 12.

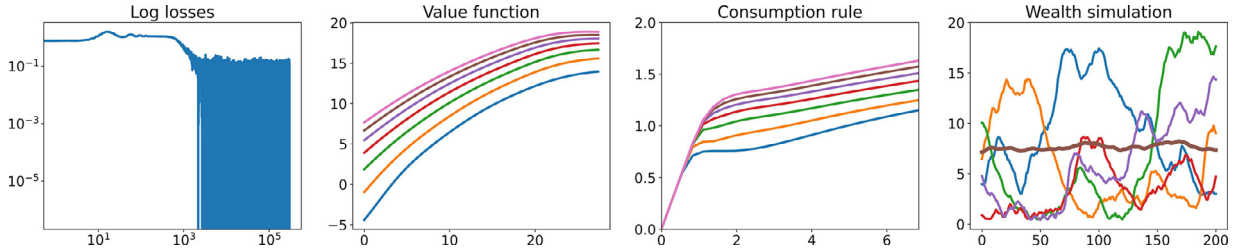


Fig. 12. Bellman-equation method in Krusell and Smith (1998) model.

To initialize the algorithm, we pre-train the value function during the first 100,000 iterations holding initial decision functions for consumption and multiplier fixed – this explains the initial flat area in the loss function. In the figure, we also plot the resulting value function under 7 different individual productivity levels ranging from -2 to 2 of the standard deviations of productivity. The constructed decision functions and simulated series look very similar to those produced by the two previous methods.

5.6. Comparison of the solutions produced by three methods

To make a more conclusive judgement, in Fig. 13, we compare the decision rules produced by the three methods.

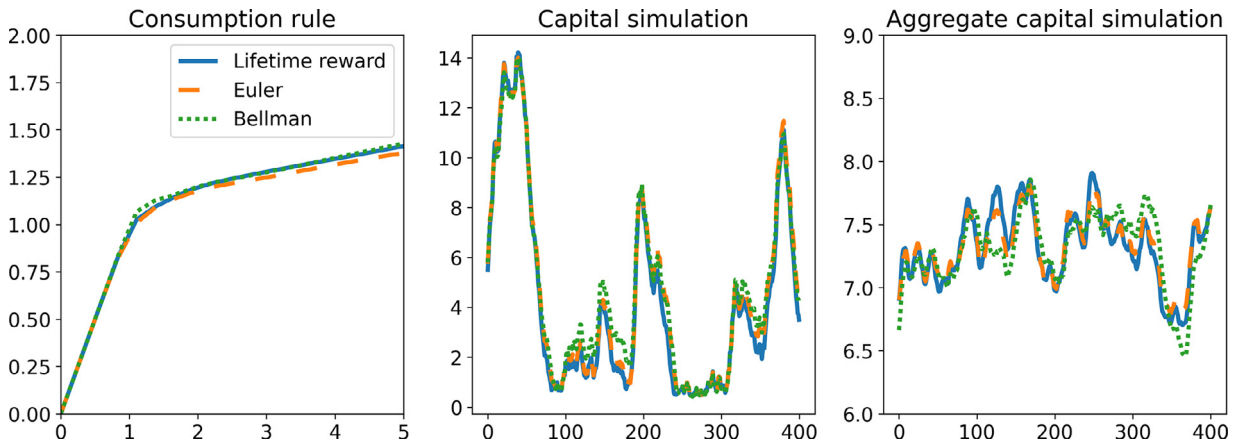


Fig. 13. Comparison of methods in Krusell and Smith (1998) model.

In the first panel, we plot the decision rules of one agent for the three methods by assuming that all other individual and aggregate state variables are in the steady state. In the second panel, we show simulation of individual wealth for all three methods under an identical sequence of shocks and in the last panel, we show simulation of aggregate wealth. The constructed decision rules and time series solutions are visually close, and they produce very similar statistics such as the first and second moments. We should take into account that some of the differences between these three solutions are explained by randomness that is innate to stochastic optimization, namely, the constructed decision rules may somewhat fluctuate along iterations depending on the specific sequence of random shocks.

5.7. Euler-equation method: The properties of the solution

In Table 1, we present some statistics for the model with different number of agents produced by the Euler-equation version of our solution method.

Table 1
Selected statistics for Krusell and Smith (1998) model.

ℓ	$std(y)$	$corr(y, c)$	$Gini(k)$	Bottom 40%	Top 20%	Top 1%	Time, sec.	R^2
1	1.69	0.862	–	–	–	–	522	0.9837
5	1.69	0.681	0.403	0.143	0.446	0.034	678	0.9910
10	1.64	0.671	0.443	0.115	0.469	0.037	805	0.9934
50	1.64	0.681	0.447	0.113	0.473	0.036	1721	0.9898
100	1.66	0.708	0.430	0.123	0.460	0.036	3297	0.9936
500	1.63	0.699	0.438	0.119	0.467	0.037	21,823	0.9965
1000	1.66	0.707	0.430	0.118	0.465	0.037	43,241	0.9977

In the first column, we show that all the studied models have practically the same standard deviation of output equal to $\text{std}(y) \approx 1.64$. This is because we normalize the mean of individual shocks to one in every period to eliminate the effect on idiosyncratic shocks on the aggregate economy. In the second column, we provide the correlation between output and aggregate consumption, which visibly decreases with the number of agents. There is literature that tries to understand why real business cycle models overstate the correlation between these two variables and our analysis suggests that the heterogeneity can be a clue.

Columns 3–6 report the Gini coefficient of the wealth distribution and the share of income by quantiles. Here, the numbers are comparable across the models and are similar to those obtained in Aiyagari (1994). This fact is not particularly surprising since our calibration closely follows the one used in that paper. We differ from that paper in that we also introduce the aggregate shocks but this is not a sufficiently strong mechanism to change the distributional implications of the baseline Aiyagari's (1994) model.

Column 7 reports the running time. We see that the time varies from 522 to 43,241 seconds which is not prohibitively large. We conclude that much larger models can be solved using a more powerful hardware beyond a laptop. In fact, the bottleneck is actually not the running time but memory: manipulating large neural networks becomes increasingly expensive as the number of agents increases.

Finally, column 8 contains the most interesting and controversial statistic which is R^2 of Krusell and Smith (1998) style regression:

$$\ln(k_{t+1}) = \xi_0 + \xi_1 \ln(k_t) + \xi_2 \ln(z_t), \quad (46)$$

i.e., a regression of aggregate capital on the past aggregate capital and aggregate productivity; see Den Haan (2010) for a discussion. Krusell and Smith (1998) find that R^2 in their model was in excess of 0.99999, which means that the aggregate capital k_{t+1} , and hence, the prices, can be accurately predicted by using just aggregate state variables k_t and z_t . This result is referred to as *approximate aggregation*. In Table 1, we see that R^2 is also relatively large, e.g., it is in excess of 0.98 for all models. However, it is not as large as the one reported by Krusell and Smith (1998) and other papers that implemented related methods, e.g., Maliar et al. (2010).

However, our analysis is not exactly identical to the one studied by Krusell and Smith (1998). They had two aggregate shocks and solve for two state-contingent rules $\ln(k_{t+1}) = \xi_0^g + \xi_1^g \ln(k_t)$ and $\ln(k_{t+1}) = \xi_0^b + \xi_1^b \ln(k_t)$, where "g" and "b" denote the good and bad aggregate-productivity states. In their state-contingent regressions, the sampling errors are associated only with the aggregate capital. We have a more complicated setup with a continuum of aggregate states. Our sampling errors in (46) are driven by both the aggregate capital and aggregate productivity. Possibly, if we split the data by the level of aggregate productivity to mimic Krusell and Smith (1998) state-contingent regressions, we would get R^2 which is closer to theirs.

5.8. Deep learning method with a reduced state space

To reduce the computational expense, Krusell and Smith (1998) came up with a simple and effective idea of replacing the distributions of state variables $D_t \equiv \{y_t^i, w_t^i\}_{i=1}^\ell$ by a set of moments m_t . To implement this idea, they designed a fixed-point iterative procedure that alternates between constructing the individual decision rules on a grid (by taking the law of motion of the moments as given) and solving for the law of motion for the moments (by taking the individual decision rules as given). An essential part of their solution method is a regression of moments on lagged moments shown in Table 1. If such regression has a high explanatory power, the agents can accurately predict future prices without knowing the distributions which reduces the individual state space to just four state variables (y_t^i, w_t^i, m_t, z_t) if only first moment is used. Krusell and Smith (1998) approach proved to work remarkably well in a variety of models and applications.¹¹

Our deep learning framework provides a simple way to incorporate Krusell and Smith (1998) idea of using a reduced state space. As before, we simulate a panel of heterogeneous agents but we feed moments in the decision functions instead of distributions. Under our implementation, there is no need to alternate between constructing the individual and aggregate solutions as we solve the entire model at once. Moreover, the regression step of Krusell and Smith (1998) analysis is also unnecessary, which allows us to use our method in those models in which the explanatory power of such regression is insufficient. Our deep learning method admits any other statistics instead of and in addition to moments and provides a cheap alternative to our baseline deep learning operating on the actual state space. Having relatively few moments, like 10 or 20, makes our method tractable even with larger number of agents, in particular, we are able to increase the number of agents to 10,000. To save on space, we do not show the results since the statistics and figures are practically the same as in the baseline case.

¹¹ Other early approaches to solving heterogeneous agent models are offered by Den Haan (1997); Judd (1998); Reiter (2010); see Den Haan (2010) for a review.

5.9. Deep learning versus moments

Krusell and Smith (1998) discovered that a single statistic – the mean of the wealth distribution – can effectively characterize the state space of their heterogeneous-agent economy. An interesting question is how their solution compares to our analysis. To address this question, we compare a solution produced when using the first moment of wealth distribution (KS1) with our baseline solution produced when using the actual state space (64×64 neurons). We find that the one-moment KS1 solution is systematically shifted up relative to our baseline 64×64 solution (although the difference is not quantitatively important). We then compute a deep learning solution with two hidden layers composed of 64 and 4 neurons, respectively, so that the second hidden layer has same dimensionality as the one-moment solution. We find that the 64×4 -neuron solution is also shifted up near the kink but it gradually approaches our reference solution for larger levels of capital; see Fig. 14.

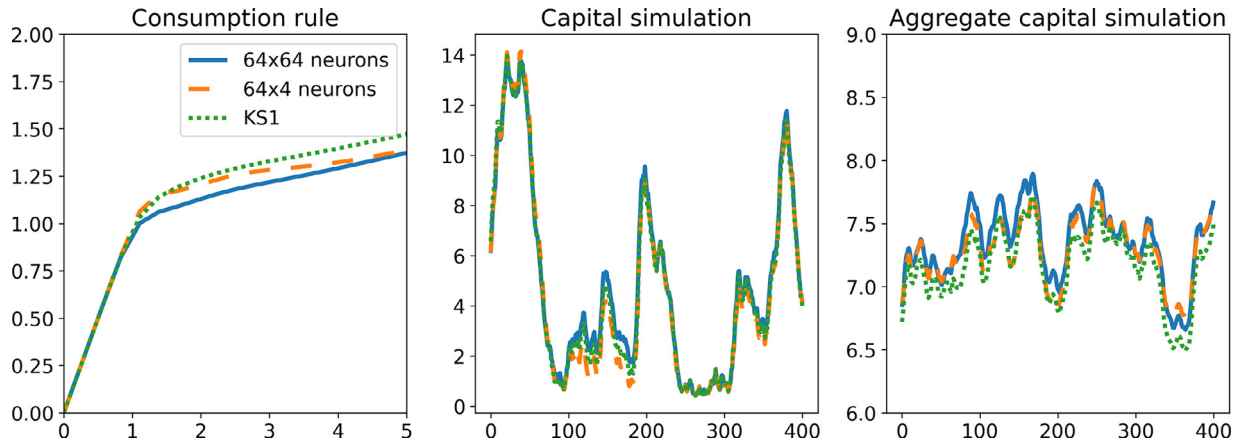


Fig. 14. Comparison of one-moment, 64×64 and 64×4 neuron solutions.

We tried to add second and third moments but it did not help remove the shift (we explore both ordinary and orthogonal Hermite polynomials). In fact, a Krusell and Smith (1998) solution constructed using two moments of the wealth distribution, KS2, was even further away from our reference 64×64 solution than the one-moment solution KS1. We next consider a solution with seven neurons in the hidden layer 64×7 which has the same dimensionality as Krusell and Smith (1998) solution with first and second moments KS2, and we find that additional neurons do help get closer to the reference solution, see Fig. 15.

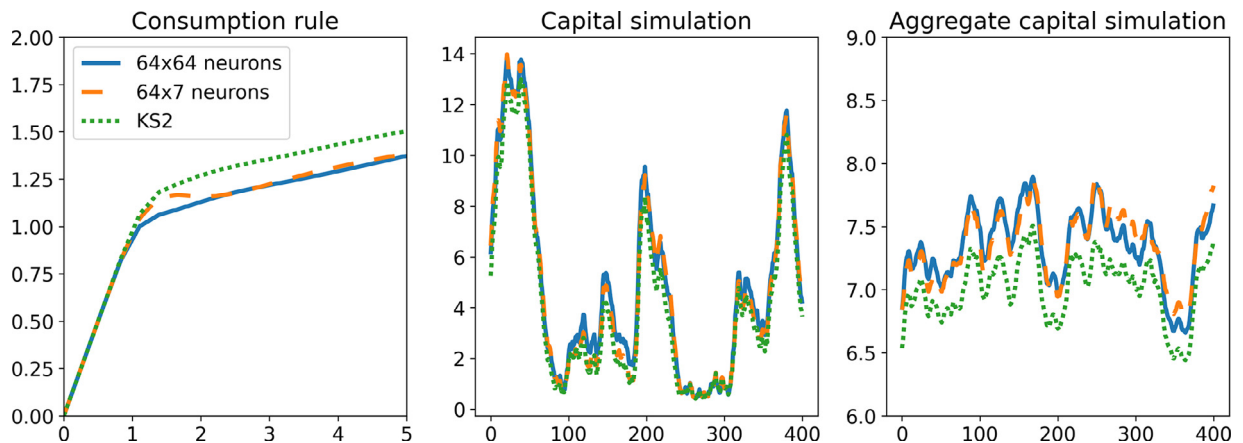


Fig. 15. Comparison of two-moment, 64×64 and 64×7 neuron solutions.

Why does adding neurons help but adding more moments does not? This is because the second moments do not have additional explanatory power relative to the first moment, for example, they do not increase R^2 of [Krusell and Smith \(1998\)](#) regression. Adding such redundant moments does not increase the fit but does increase the variance. In contrast, neural network is designed to perform model reduction: it considers many possible ways of extracting the information contained in the distribution and condensing it into a relatively small set of hidden layers. Not surprisingly, it can find representations that are more efficient than a set of moments postulated ad-hoc and it can use additional neurons to improve on the quality of representation.

6. Discussion: Deep learning in economic dynamics

In this section, we discuss a broad set of issues related to applying deep learning in economic dynamics and explain how these issues differ from those arising in data science.

The main novelty: computational technology! There is not much methodological novelty in the canonical AI analysis. The gradient descent method is known since Newton and its stochastic optimization version was developed in early 1950s. Neural networks were also discovered long ago ([Rosenblatt \(1958\)](#)). There are even remarkable early applications of neural networks to solving dynamic economic models ([Duffy and McNelis \(2001\)](#)).

However, machine learning is so technology intensive that DL methods were put aside until platforms like TensorFlow and Pytorch were developed to facilitate their implementation. Google has developed its own TPU units too. High-end GPUs feature thousands of powerful CUDA cores, which can all operate at the same time. Also, commercial interest is high and the surge of cloud computing has made it possible to rent vast amounts of computing power, e.g., Amazon.

Learning is always deep in economic models. In data science, learning is called "deep" because neural networks have interconnected topologies with multiple layers of coefficients. In economic dynamics, there is another reason for calling learning "deep". Specifically, objective functions derived from dynamic economic models contain variables of multiple periods and nested decision functions which lead to multilayer interconnected topologies and approximation coefficients buried in several layers of intertemporal optimization, similar to those observed in multilayer neural network. In that sense, learning is always "deep" in dynamic economic models, even for simple one-layer approximating functions such as polynomial or piecewise linear functions.

Data are truly random in economic dynamics. In data science, a data set is usually fixed and batches are not really random but a pseudo-random bootstrap of the given data. In such applications, we split the available data into 3 samples, namely, for construction of a solution, validation and accuracy assessment. When solving dynamic models, we do not use real-world economic data but generate artificial data by simulating the model. In this case, we just need to fix the distribution for random draws. We then are able to simulate the model at will and we can generate as much artificial data as we want. In that sense, our data are truly random.

Antithetic variates improve efficiency of Monte Carlo integration. Monte Carlo integration has a low square-root rate of convergence. We can increase the efficiency of SGD by using variance reduction techniques; see [Cheng \(1982\)](#). One simple method is antithetic variates: assuming a zero mean, for every realization $(\omega_1, \dots, \omega_{n'})$, we also consider its antithetic realization $(-\omega_1, \dots, -\omega_{n'})$. Another possibility is a tensor-product antithetic variates, i.e., to consider all possible combinations $(\pm\omega_1, \dots, \pm\omega_{n'})$. In the lifetime reward function, the sequence $(\pm\epsilon_1, \dots, \pm\epsilon_T)$ may be expensive to analyze, so we can consider a truncated sequence with antithetic variates just for the first τ periods, namely, $(\pm\epsilon_1, \dots, \pm\epsilon_\tau, \epsilon_{\tau+1}, \dots, \epsilon_T)$. The distant future terms are discounted so that making the first few draws antithetic can still bring a considerable increase in accuracy.

Lifetime reward maximization with deterministic integration of shocks. For the Euler- and Bellman-equation methods, we had two versions: one in which we compute integrals with respect to state variables and future shocks separately and the other is the AiO expectation version. For the lifetime reward, we only have the version with the AiO expectation operator. But we can also implement a method that computes the two expectation functions separately, for example, one that uses Monte Carlo integration for evaluating $E_{(m_0, s_0)}[\cdot]$, and the other that uses deterministic integration for evaluating $E_{(\epsilon_1, \dots, \epsilon_T)}$, i.e., we compute sequentially the two expectation operators in $E_{(m_0, s_0)}[E_{(\epsilon_1, \dots, \epsilon_T)}[\sum_{t=0}^T \beta^t r(\cdot)]]$. To evaluate $E_{(\epsilon_1, \dots, \epsilon_T)}$, we can use a deterministic integration method suggested in [Adjemian and Juillard \(2013\)](#). In each period t , we construct n integration nodes (by using quadrature, monomials, etc.) This leads to an exploding tensor-product tree. For example, a tree with just 2 Gaussian nodes $\pm\epsilon$, results in sequence $\pm\epsilon_0, \pm\epsilon_1, \pm\epsilon_2, \dots$ that has exponentially growing number of nodes 2, 4, 8, But [Adjemian and Juillard \(2013\)](#) propose a clever refinement that makes the problem tractable by eliminating those branches of the tree whose probabilities are low.

Nested local approximations and deterministic models. Each of the three constructed objective functions contains two nested models: One is the model in which the solution is approximated locally around the given state, and the other is the deterministic model. In the former case, state (m, s) is fixed in [\(8\)](#), [\(10\)](#), [\(12\)](#), [\(14\)](#), and [\(15\)](#) and expectation is computed

only with respect to exogenous shocks ϵ . Such a solution may be interesting per se, for example, for studying transitions of a developing economy to the steady state because the solution constructed just in the ergodic set may be insufficiently accurate. In turn, the deterministic model is the one in which a realization of shocks ϵ is fixed and the state (m, s) is random.

Accuracy tests in economic dynamics are indirect. In a canonical supervised learning regression, we assess the quality of approximation by looking at the difference between the true and predicted output out-of-sample. In a logistic regression (e.g., the problem of handwritten digits classification), the success can be also measured by the fraction of times the machine classifies the digits correctly, e.g., it recognizes the handwritten digits correctly in 80 percent of the cases.

In the context of economic dynamics, a parallel direct accuracy test would require us to compare approximate and exact solutions. This is generally infeasible since the exact solutions are unknown. One way to deal with this complication is to construct a more accurate reference solution by using more flexible approximation functions, more precise solvers and more accurate numerical integration methods. However, such a reference solution may be infeasible or excessively costly; see Judd et al. (2017) for a discussion and examples of cheaper direct testing methods that rely on numerical construction of the error bounds.

Following the economic literature, we concentrate on indirect approaches to the accuracy evaluation. Specifically, we will check certain properties that an accurate solution is known to satisfy, such as zero residuals in the Euler or Bellman equation. Indirect accuracy tests are simple to design and they can be implemented in an out-of-sample way which is characteristic for AI applications. Moreover, we can define indirect accuracy measures to reflect the economic significance of accuracy. For example, we can express approximation errors in percentage terms of consumption.

7. Concluding comments

In the paper, we propose an AI technology that is tractable in large scale applications – a deep learning method based on Monte Carlo simulation. Our analysis is technology driven: we do not aim to design AI approaches that would work best for a certain class of economic models but rather we adapt the economic models themselves to available AI technologies. The modern data-science tools are ubiquitous, well developed, free of errors and optimized – these may be sufficient to compensate for potential inefficiencies. We have shown the promise of the DL approach by solving the Krusell and Smith (1993) model with thousands of state variables without resorting to a simplifying assumption about the economy's state space – such analysis has been infeasible up to now. Consequently, it seems to be a promising direction to explore.

Our solution framework was designed to take advantage of existing DL technology. "Is this the best possible technology for solving dynamic economic models?" – the answer to this question is not clear.

First, neural networks are powerful universal approximators, but their training is expensive and their convergence to the solutions is not guaranteed. It is actually an open question whether there is much value in using deep neural networks for approximating decision rules in economics which often can be well approximated by simple functions like polynomials and splines.

Second, Monte Carlo simulation lying at the basis of DL framework has a low square-root rate of convergence. It is possible to improve on the Monte Carlo method by engineering sequences that deliver more accurate approximations to integrals (e.g., quadrature, monomials, quasi-random sequence, sparse grids, clusters, epsilon-distinguishable sets), as well as by applying variance-reduction techniques such as antithetic variates; see Maliar et al. (2014) for a review.

Third, instead of stochastic optimization, we can use other numerical solvers, e.g., fixed-point iteration, conventional GD methods, Gauss-Jacobi, Gauss-Siedel and linear programming; see Judd (1998). These techniques are commonly used in computational economics, and we expect them to be useful alternatives to our baseline SGD in some applications.

Finally, there are other AI-style methods that can be used for solving economic models, in particular, unsupervised and reinforcement learning methods. These methods offer a possibility of online learning and additional powerful approximation techniques such as alternating of learning, exploration or exploitation – such techniques are absent in our static offline supervised learning framework.

Acknowledgements

Lilia Maliar and Serguei Maliar acknowledge financial support from the NSF grants SES-1949413 and SES-1949430, respectively. We are grateful to Marc Maliar for his help with writing the TensorFlow code for solving Krusell and Smith (1998) model. We thank the editor and an anonymous referee for many useful comments and suggestions. The paper also circulated under the title "Will Artificial Intelligence Replace Computational Economists Any Time Soon?" We also received useful comments from participants of the 2018 Society for Computational Economics (CEF) Conference in Milan (invited session), 2018 Econometric Society Australasian Meeting (ESAM) in Auckland (invited talk), Oxford University, CEPREMAP, Banque de France, Panorsk Summer School, NYU Abhu Dabi, Paris Dauphine University, CREST (Ecole Polytechnique), Durham University, Santa Clara University, Rutgers University, PASC conference, Model comparison conference in Frankfurt, Complutense University of Madrid, Stanford University, Queens College, Stony Brook University, Columbia University, University College of London, Deutsche Bundesbank, the Graduate Center, CUNY.

Supplementary material

Supplementary material associated with this article can be found, in the online version, at doi:[10.1016/j.jmoneco.2021.07.004](https://doi.org/10.1016/j.jmoneco.2021.07.004).

References

- Adjemian, S., Juillard, M., 2013. Stochastic extended path approach. Manuscript.
- Ahn, S., Kaplan, G., Moll, B., Winberry, T., Wolf, C., 2018. When inequality matters for macro and macro matters for inequality. NBER Macroeconomics Annual, University of Chicago Press, vol. 32 (1), 1–75.
- Arellano, C., Maliar, L., Maliar, S., Tsyrennikov, V., 2016. Envelope condition method with an application to default risk models. Journal of Economic Dynamics and Control 69, 436–459.
- Aruoba, S.B., Fernández-Villaverde, J., Rubio-Ramírez, J., 2006. Comparing solution methods for dynamic equilibrium economies. Journal of Economic Dynamics and Control 30, 2477–2508.
- Azinovic, M., Luca, J., Scheidegger, S., 2020. Deep equilibrium nets. SSRN: <https://ssrn.com/abstract=3393482>
- Bayer, C., Luetticke, R., 2020. Solving discrete time heterogeneous agent models with aggregate risk and many idiosyncratic states by perturbation. Quant Econom 11, 1253–1288.
- Carroll, C., 2006. The method of endogenous gridpoints for solving dynamic stochastic optimization problems. Econ Lett 91 (3), 312–320.
- Cheng, R., 1982. The use of antithetic variates in computer simulations. Journal of the Operational Research Society 33 (3), 229–237.
- Coleman, C., Lyon, S., Maliar, L., Maliar, S., 2018. Matlab, python, julia: what to choose in economics? CEPR working paper DP 13210. Computational Economics, forthcoming.
- Den Haan, W., 1997. Solving dynamic models with aggregate shocks and heterogeneous agents. Macroecon Dyn 1 (02), 355–386.
- Den Haan, W., 2010. Comparison of solutions to the incomplete markets model with aggregate uncertainty. Journal of Economic Dynamics and Control 34, 4–27.
- Den Haan, W., Marcet, A., 1990. Solving the stochastic growth model by parameterized expectations. Journal of Business and Economic Statistics 8, 31–34.
- Duarte, V., 2018. Machine learning for continuous-time economics. SSRN: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3012602
- Duffy, J., McNelis, P., 2001. Approximating and simulating the real business cycle model: parameterized expectations, neural networks, and the genetic algorithm. Journal of Economic Dynamics and Control 25 (9), 1273–1303.
- Fernández-Villaverde, J., Hurtado, S., Nuño, G., 2019. Financial frictions and the wealth distribution. NBER Working paper 26302.
- Goodfellow, I., Bengio, Y., Courville, A., 2016. Deep learning. Massachusetts Institute Technology Press.
- Gorodnichenko, Y., Maliar, L., Maliar, S., Naubert, C., 2020. Household savings and monetary policy under individual and aggregate stochastic volatility. CEPR working paper 15614.
- Jiang, H., 1996. Smoothed fischer-burmeister equation methods for the complementarity problem. Manuscript.
- Jirniy, A., Lepetyuk, V., 2011. A reinforcement learning approach to solving incomplete market models with aggregate uncertainty. SSRN: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=1832745
- Judd, K.L., 1992. Projection methods for solving aggregate growth models. J Econ Theory 58, 410–452.
- Judd, K.L., 1998. Numerical methods in economics. MIT Press.
- Judd, K.L., Maliar, L., Maliar, S., 2011. Numerically stable and accurate stochastic simulation approaches for solving dynamic models. Quant Econom 2, 173–210.
- Judd, L., Maliar, L., Maliar, S., 2017. Lower bounds on approximation errors to numerical solutions of dynamic economic models. Econometrica 85 (3), 991–1020.
- Krusell, P., Smith, A., 1998. Income and wealth heterogeneity in the macroeconomy. Journal of Political Economy 106, 868–896.
- Lepetyuk, V., Maliar, L., Maliar, S., 2020. When the u.s. catches a cold, canada sneezes: a lower-bound tale told by deep learning. Journal of Economic Dynamics and Control 117, 103926.
- Maliar, L., Maliar, S., 2005. Parameterized expectations algorithm: how to solve for labor easily. Computational Economics 25, 269–274.
- Maliar, L., Maliar, S., 2013. Envelope condition method versus endogenous grid method for solving dynamic programming problems. Econ Lett 120, 262–266.
- Maliar, L., Maliar, S., 2015. Merging simulation and projection approaches to solve high-dimensional problems with an application to a new keynesian model. Quant Econom 6, 1–47.
- Maliar, L., Maliar, S., 2020. Deep learning classification: modeling discrete labor choice. CEPR working paper DP 15346.
- Maliar, L., Maliar, S., 2014. Numerical Methods for Large Scale Dynamic Economic Models. In: Schmiedders, K., Judd, K. (Eds.), Handbook of Computational Economics, Volume 3, Chapter 7. Elsevier Science, Amsterdam, pp. 325–477.
- Maliar, L., Maliar, S., Valli, F., 2010. Solving the incomplete markets model with aggregate uncertainty using the krusell-smith algorithm. Journal of Economic Dynamics and Control 34, 42–49.
- Reiter, M., 2010. Approximate and almost-exact aggregation in dynamic stochastic heterogeneous-agent models. IHS Working Paper 258.
- Rosenblatt, F., 1958. The perceptron: a probabilistic model for information storage and organization in the brain. Psychol Rev 65 (6), 386–408.
- Rust, J., Amman, H., Kendrick, D., Rust, J., 1996. Numerical Dynamic Programming in Economics. In: Handbook of Computational Economics. Elsevier Science, Amsterdam, pp. 619–722.
- Santos, M., Taylor, J., Woodford, M., 1999. Numerical Solution of Dynamic Economic Models. In: Handbook of Macroeconomics. Elsevier Science, Amsterdam, pp. 312–382.
- Smith, A., 1993. Estimating nonlinear time-series models using simulated vector autoregressions. Journal of Applied Econometrics 8, S63–S84.
- Stachurski, J., 2009. Economic dynamics: Theory and computation. MIT Press.
- Villa, A., Valaitis, V., 2019. Machine learning projection methods for macro-finance models. SSRN: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3209934
- Winberry, T., 2018. A method for solving and estimating heterogeneous agent macro models. Quant Econom 9 (3), 1123–1151.